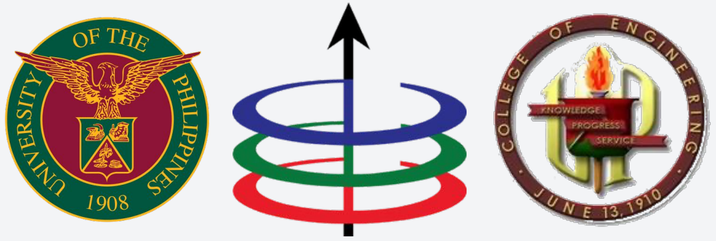


Lecture 2A
Algorithm Analysis - Big O

EEE 121 2s2526

Steven S. Sison

- All about algorithms
- Runtime Analysis
- Asymptotic Notations
- Big-O

- **All about algorithms**
- Runtime Analysis
- Asymptotic Notations
- Big-O

We want to solve real world problems with algorithms

In general, the process of problem solving is divided into two parts: designing and analysis.

We want to solve real world problems with algorithms

In general, the process of problem solving is divided into two parts: designing and analysis.

Design

- Understand the goal
- Select the tools
- Compose functions to form a process

We want to solve real world problems with algorithms

In general, the process of problem solving is divided into two parts: designing and analysis.

Design

- Understand the goal
- Select the tools
- Compose functions to form a process

Analysis

- How does the process work?
- Is it correct?
- Is it efficient?

What is an algorithm?

- Series of steps or instructions that are followed in order to solve a real world problem.
- Presented in multiple ways: pseudocode, flowchart, code.

What is an algorithm?

- Series of steps or instructions that are followed in order to solve a real world problem.
- Presented in multiple ways: pseudocode, flowchart, code.

Example: Euclid's Algorithm

In Pseudocode

- Let `a` be the larger integer and `b` be the smaller
- Solve for `(a%b)`.
- If the remainder is not zero, solve for `gcd(b, a%b)`.
- Otherwise, return `a`.

In C++

```
// GCD of a and b
int gcd(int a, int b){
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
```

Designing an Algorithm

When designing an algorithm, you should keep in mind these aspects:

1. **Inputs**
   a. What are the quantities or variables needed for your function to work?
2. **Outputs**
   a. What is/are the result/s of your function?
3. **Definiteness**
   a. The instructions are clear and unambiguous.
4. **Finiteness**
   a. For all cases, the algorithm should be completed in finite steps.
5. **Effectiveness**
   a. The algorithms should be simple and feasible.

Algorithm Analysis

The fundamental goal of algorithm analysis is to determine its:

- Correctness (**Algorithm Proving**. This will be next week!)
- Efficiency in terms of runtime and memory, regardless of any input size (**Time and Space Complexity**. Today!)

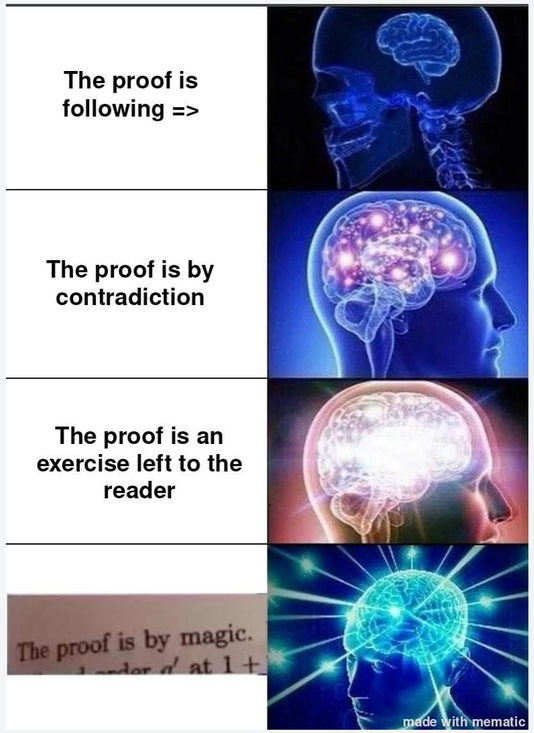
Algorithm Analysis

The fundamental goal of algorithm analysis is to determine its:

- Correctness (**Algorithm Proving**. This will be next week!)
- Efficiency in terms of runtime and memory, regardless of any input size (**Time and Space Complexity**. Today!)

Proving

- Proof by Contradiction
- Proof by Induction
- Etc. Boring maths :(



Algorithm Analysis

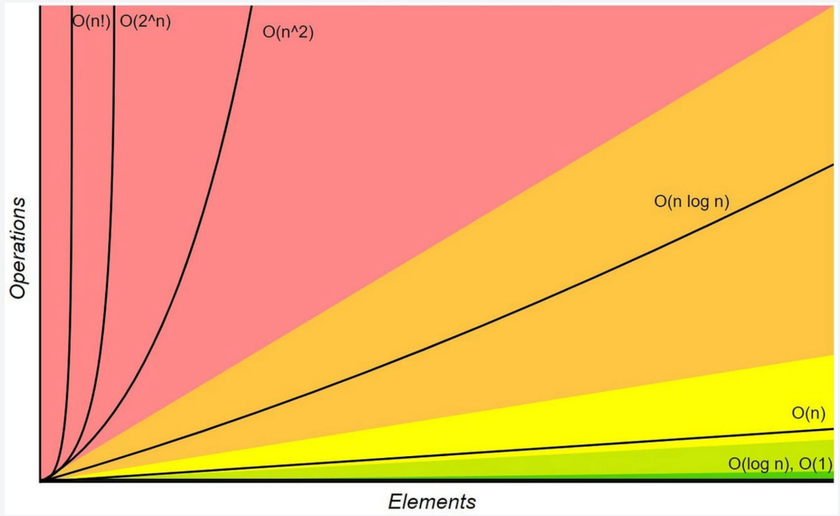
The fundamental goal of algorithm analysis is to determine its:

- Correctness (**Algorithm Proving**. This will be next week!)
- Efficiency in terms of runtime and memory, regardless of any input size (**Time and Space Complexity**. Today!)

Proving

- Proof by Contradiction
- Proof by Induction
- Etc. Boring maths :(

Time and Space Complexity



- Describes how runtime and space correlates to input size.
- Magic lines and graphs :O

Algorithm Analysis

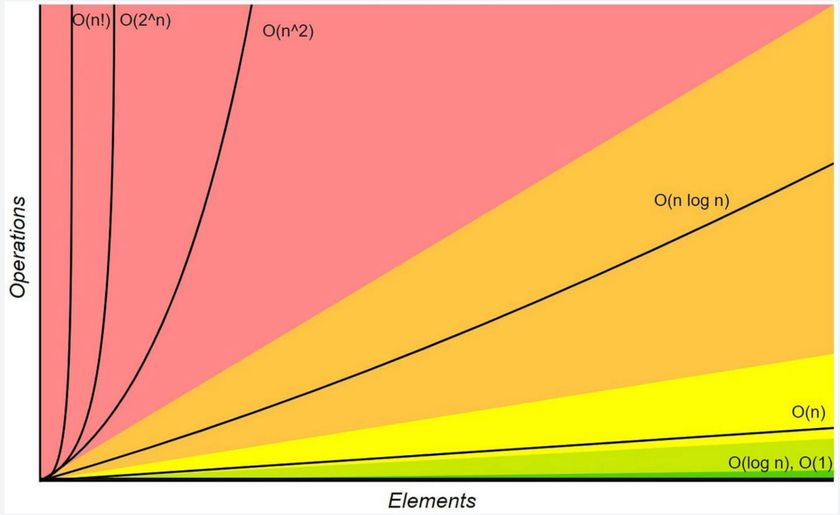
The fundamental goal of algorithm analysis is to determine its:

- Correctness (**Algorithm Proving**. This will be next week!)
- Efficiency in terms of runtime and memory, regardless of any input size (**Time and Space Complexity**. Today!)

Proving

- Proof by Contradiction
- Proof by Induction
- Etc. Boring maths :(

Time and Space Complexity



- Describes how runtime and space correlates to input size.
- Magic lines and graphs :O

- All about algorithms
- **Runtime Analysis**
- Asymptotic Notations
- Big-O

Defining “time” in Runtime

Intuitively, we define runtime as the **time it takes to run a program**.

- However, this value varies significantly.
- It is heavily dependent on the computer hardware, programming language, etc.

We need to define a universal measure for runtime

Defining “time” in Runtime

Intuitively, we define runtim[illegible]he **time it takes t[illegible] a program**.

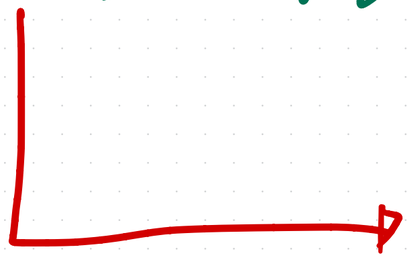
- However, this value varies sig[illegible]
- It is heavily dependent on th[illegible]programming language, etc.

We need to [illegible]ine a universal measu[illegible]e for runtime

Focus on how the runtime scales with the input size (n)

- This is much better since this is not dependent on computer hardware and any other considerations.
- However, it is only effective if n is larger compared to any constant values (we will see this later)

```
for (int i = 0; i < n; i++){
    cout << endl;
}
```



n input size	
n = 1	1
n = 10	10
n = 100	100

Types of Algorithm Analysis

The complexity of an algorithm can be analyzed in different scenarios:

1. **Best Case**
    a. This is the case where the algorithms performs fastest using the best possible input. There are **very few** times that this will happen.
    b. Example: Passing an already sorted array to a sorting algorithm.
2. **Worst Case**
    a. This is the runtime of the algorithm for the worst possible input.
    b. We want to focus our analysis on this to tell us that our algorithm is fast for any possible input
3. **Average Case and Amortized** (not in our scope, for CoE 163/164 students)

Types of Algorithm Analysis

The complexity of an algorithm can be analyzed in different scenarios:

1. **Best Case**
   a. This is the case where the algorithms performs fastest using the best possible input. There are **very few** times that this will happen.
   b. Example: Passing an already sorted array to a sorting algorithm.
2. **Worst Case**
   a. This is the runtime of the algorithm for the worst possible input.
   b. We want to focus our analysis on this to tell us that our algorithm is fast for any possible input
3. **Average Case and Amortized** (not in our scope, for CoE 163/164 students)

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

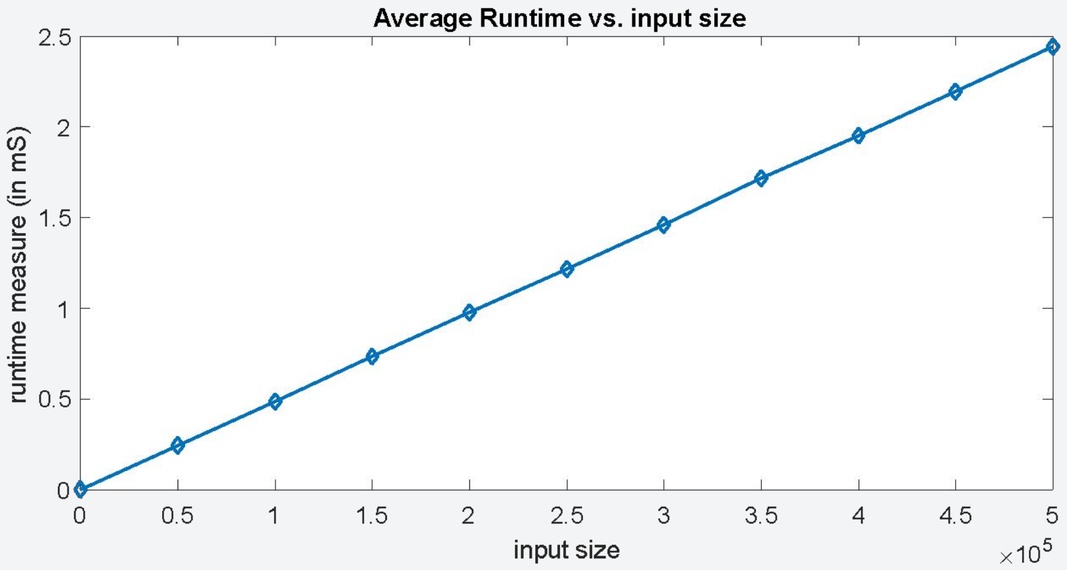
```
int get_sum(const int* arr,int n) {
 int total = 0;
 for(int i = 0; i < n; i++) {
   total += arr[i];
 }
 return total;
}
```

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

```
int get_sum(const int* arr,int n) {
 int total = 0;
 for(int i = 0; i < n; i++) {
   total += arr[i];
 }
 return total;
}
```

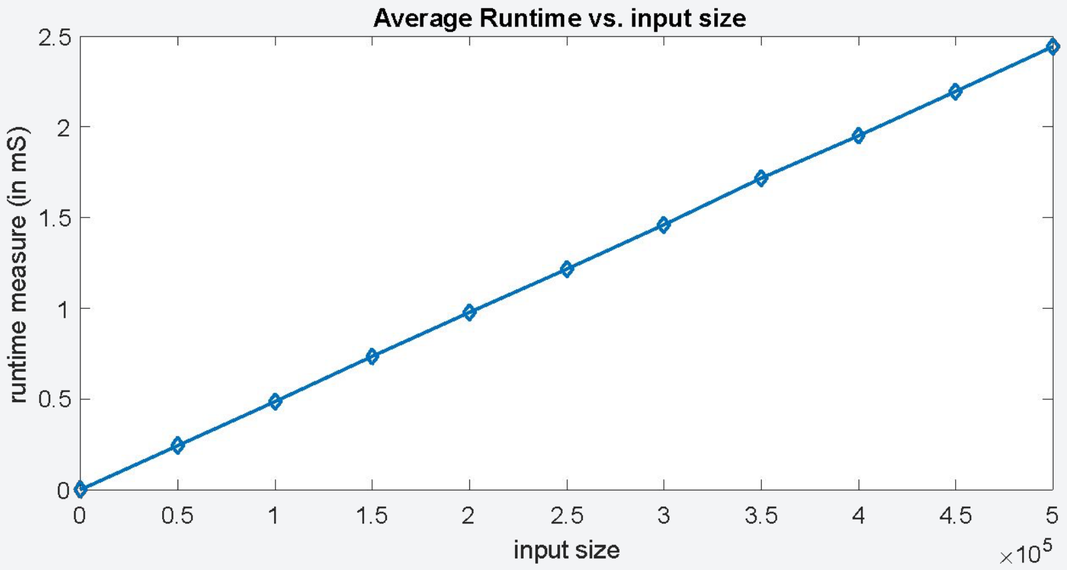


A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

```
int get_sum(const int* arr,int n) {
 int total = 0;
 for(int i = 0; i < n; i++) {
   total += arr[i];
 }
 return total;
}
```



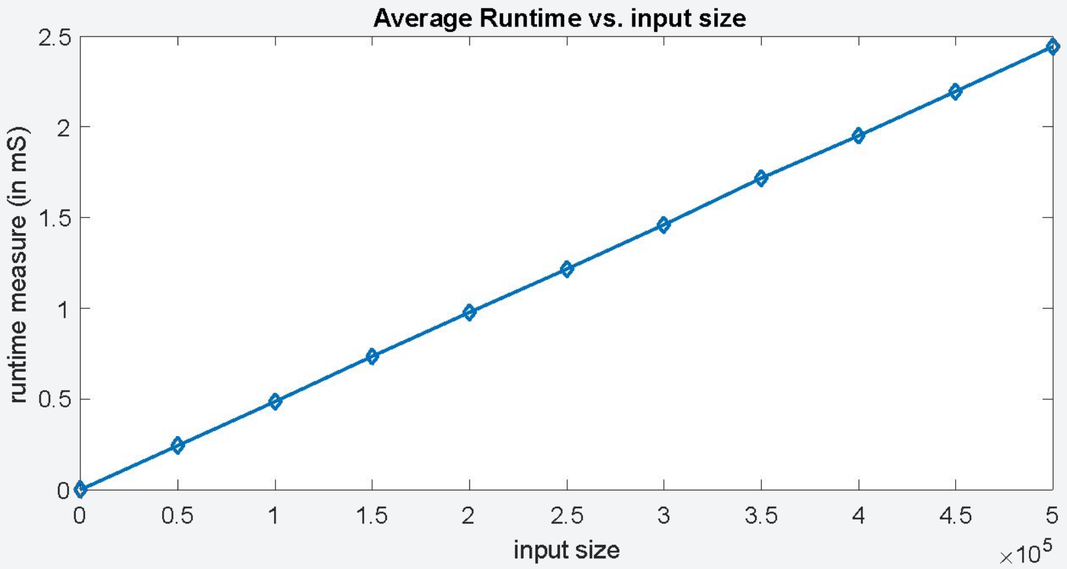
Notice anything particular?

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

```
int get_sum(const int* arr,int n) {
 int total = 0;
 for(int i = 0; i < n; i++) {
   total += arr[i];
 }
 return total;
}
```



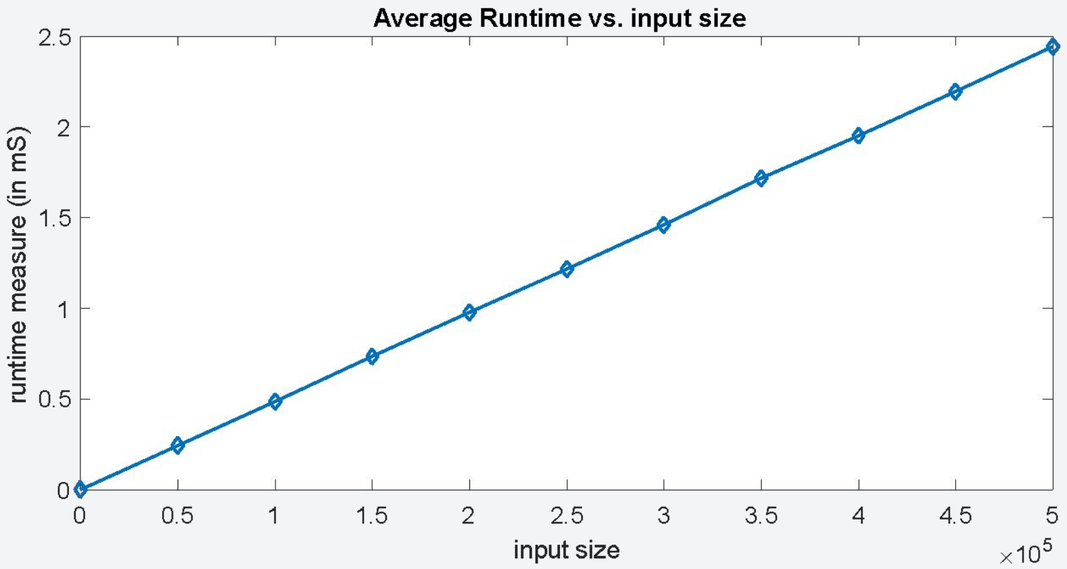
As the input size increases, so does the runtime but the values will be inconsistent among other devices.

A simple example

The worst case scenario for most algorithms when we look at the runtime as a function of n, is when $n \to \infty$ or when n is very large.

Example: A C++ function that calculates the sum of n elements of an array input.

```
int get_sum(const int* arr,int n) {
 int total = 0;
 for(int i = 0; i < n; i++) {
   total += arr[i];
 }
 return total;
}
```



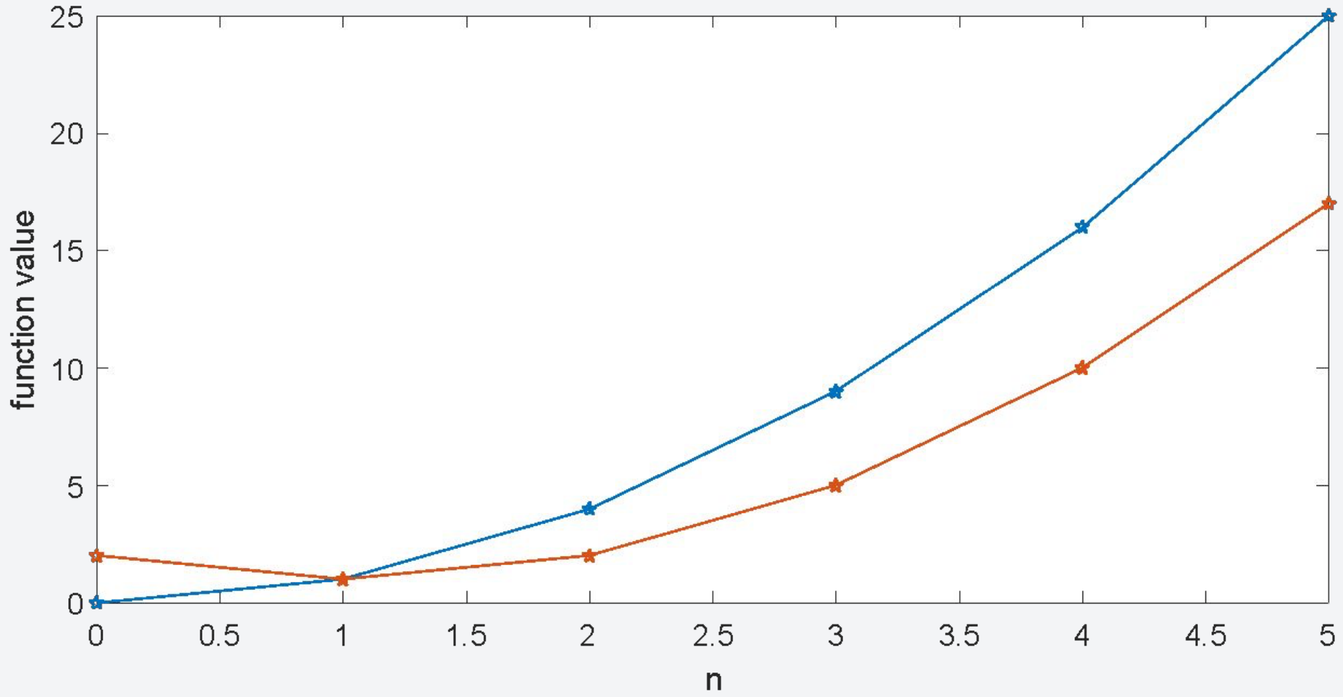
It is better to represent runtime as a function of n wherein the functions counts the number of operations instead of runtime.

- All about algorithms
- Runtime Analysis
- **Asymptotic Notations**
- Big-O

Comparing Runtimes

We've established that we will try to mathematically describe **runtime** as a **function of input size, n**. Consider the example:

$$f(n) = n^2 \text{ and } g(n) = n^2 - 2n + 2$$

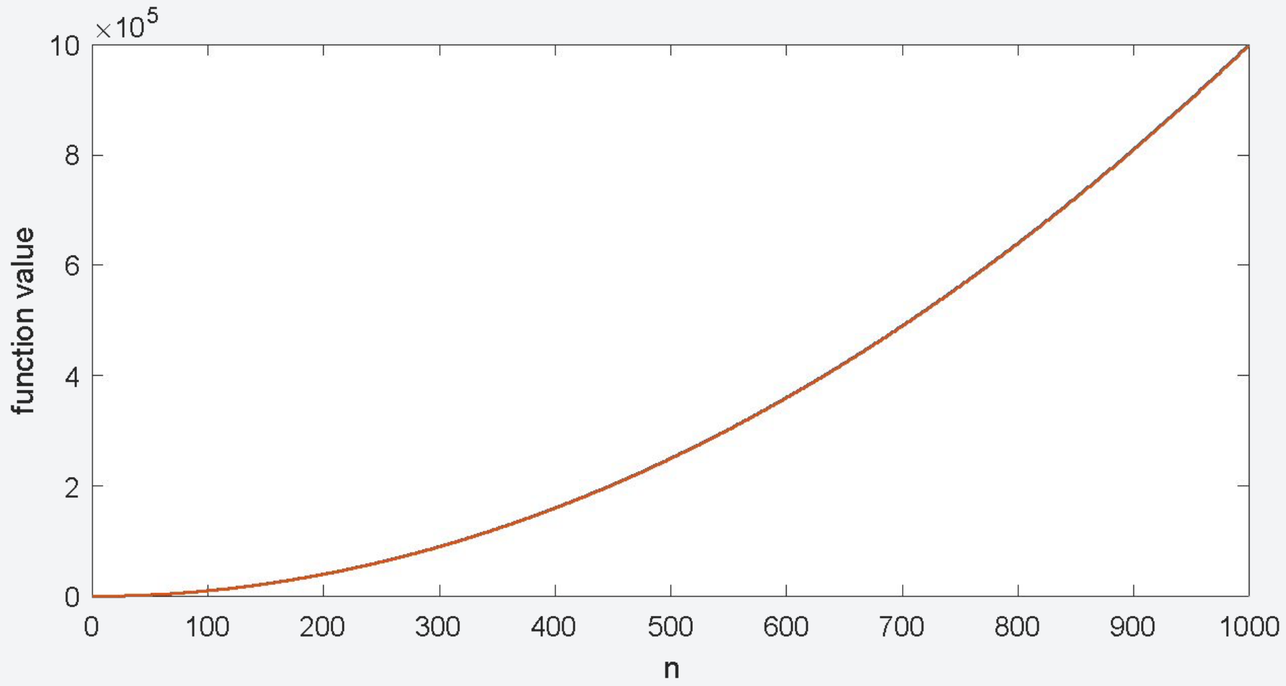


- At smaller values of n, they look very different.

Comparing Runtimes

We've established that we will try to mathematically describe **runtime** as a **function of input size, n**. Consider the example:

$f(n) = n^2$ and $g(n) = n^2 - 2n + 2$ $\Rightarrow O(n^2)$



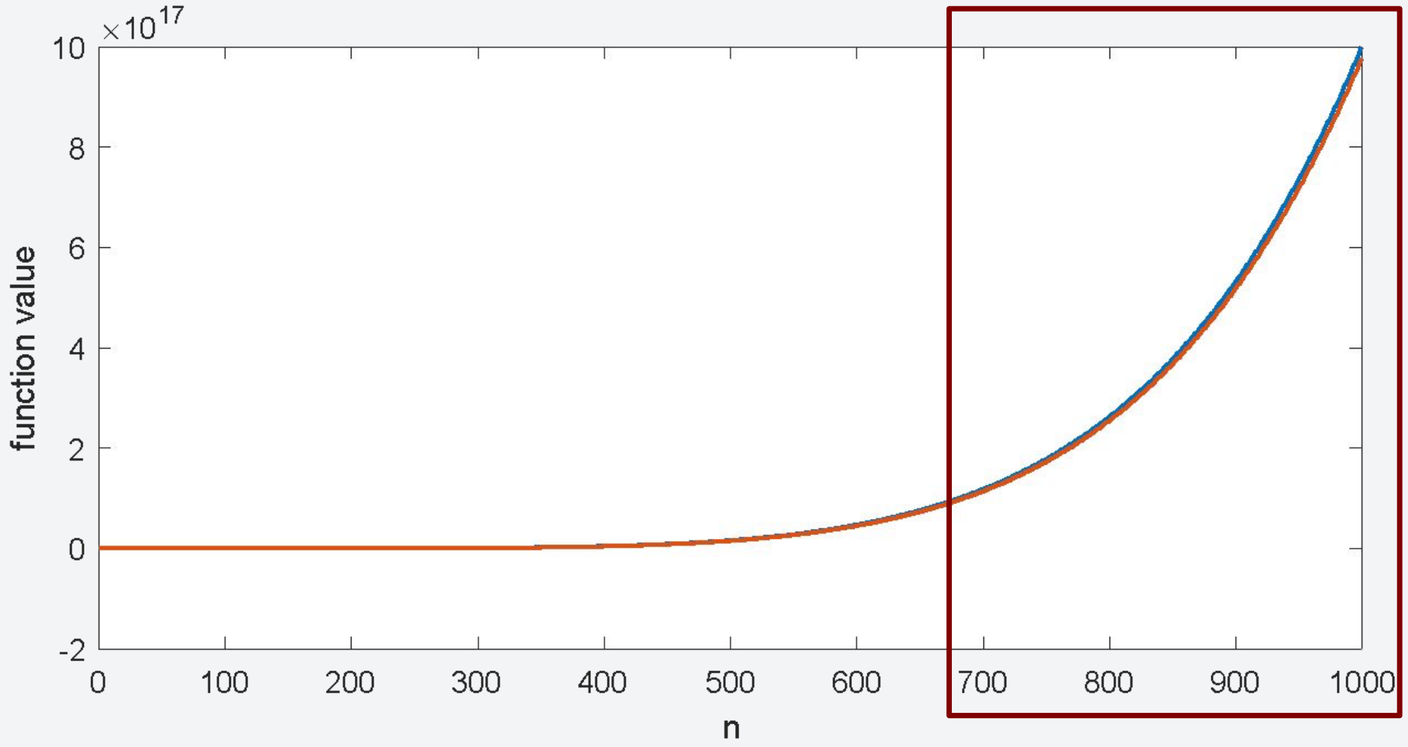
- At smaller values of n, they look very different.
- But as we zoom out, there's barely any difference between the functions.
- The relative error is very small:

$$\left|\frac{f(1000) - g(1000)}{f(1000)}\right| = 0.001998 < 0.2\%$$

Comparing Runtimes

The same thing is observed when we compare another pair of functions as long as they have the same degree.

$f(n) = n^6$ and $g(n) = n^6 - 23n^5 + 193n^4 - 729n^3 + 1206n^2 - 648n$



The difference is very small and negligible.

Asymptotic Behavior

That behavior of a function is what we call the **asymptotic behavior**.

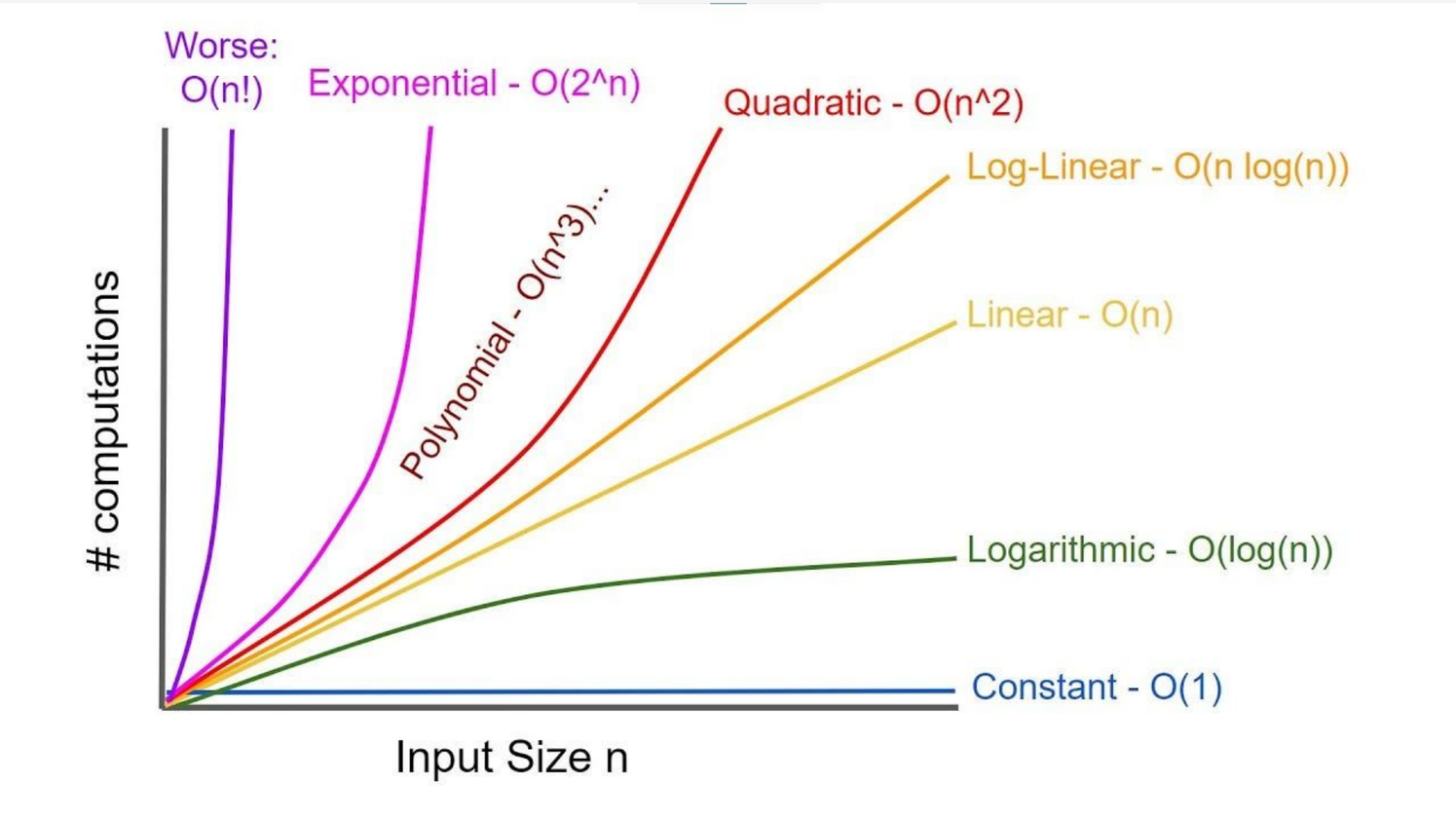
- Only the **fastest growing term (highest degree term)** matters for large input size.
- Functions of the **same degree exhibit the same rate of growth** (even for functions with different coefficients for the leading term.

These means that if we have runtime function: $f(n)=n^2$, $g(n) = 2n^2$, and $h(n)=n^3$:

- $f(n)$ and $g(n)$ will have the same time complexity.
- $h(n)$ will have a worse time complexity than both $f(n)$ and $g(n)$

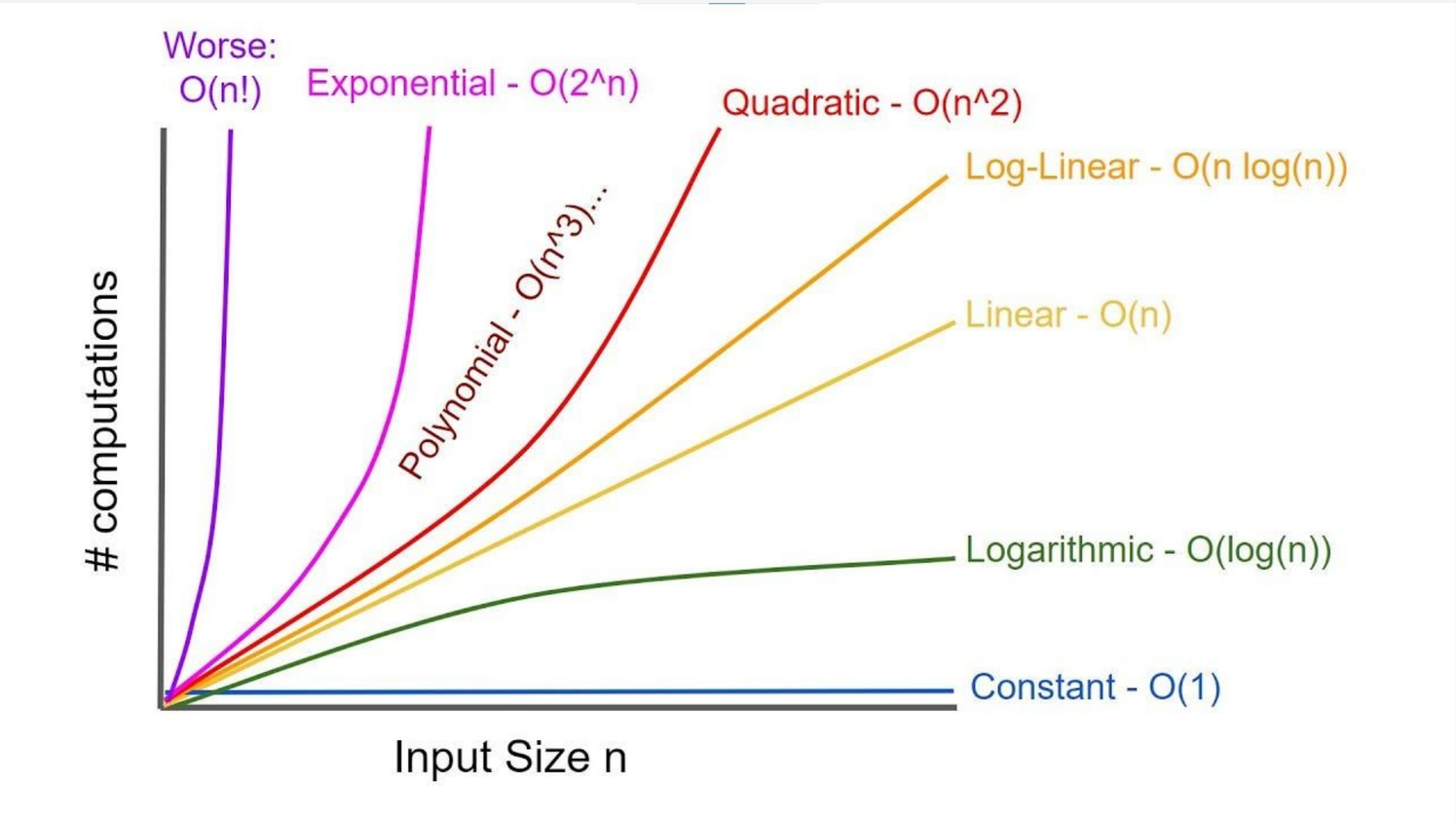
Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:



Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:

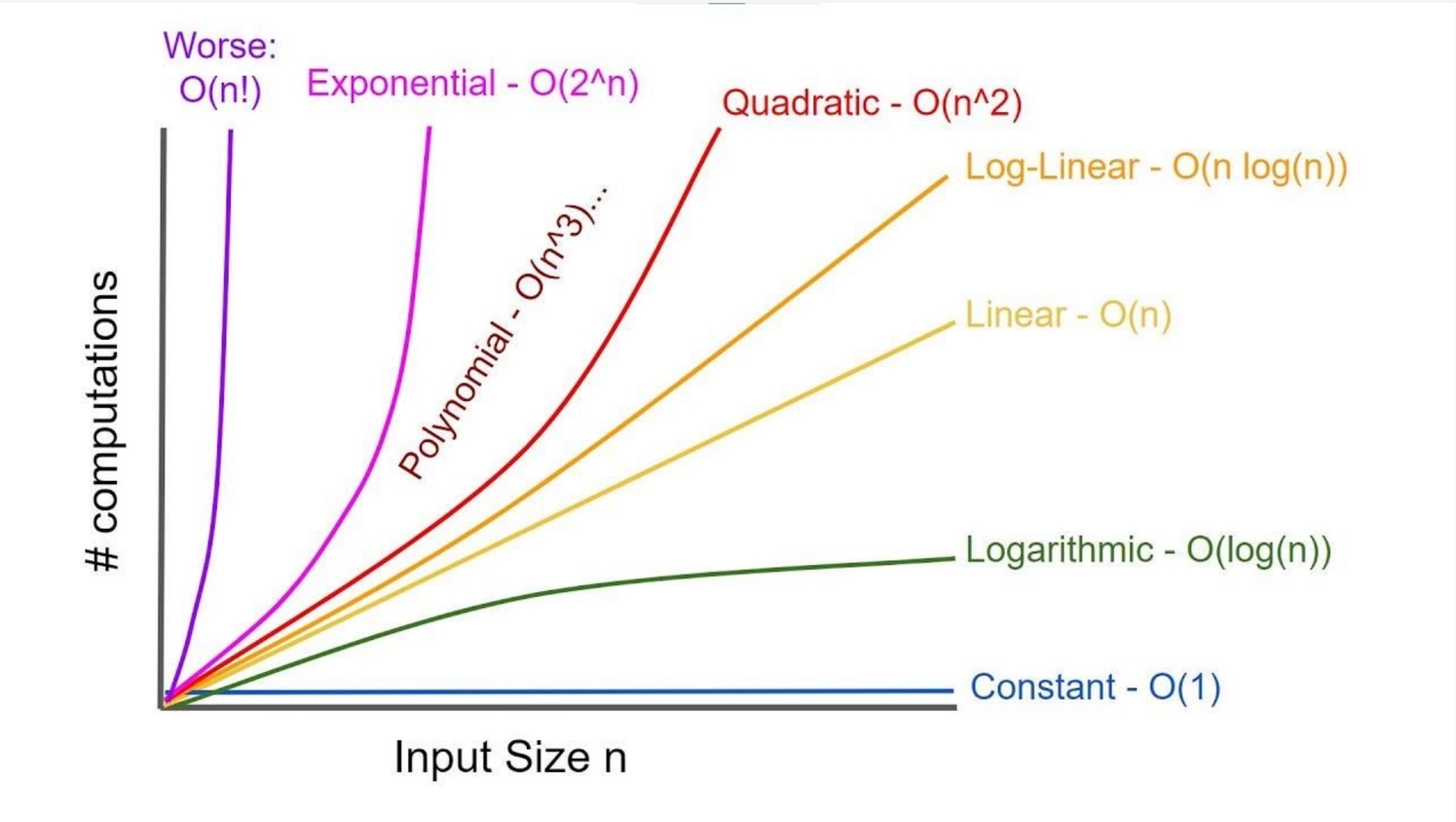


For example:

$$T_1(n) = An + B$$

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:



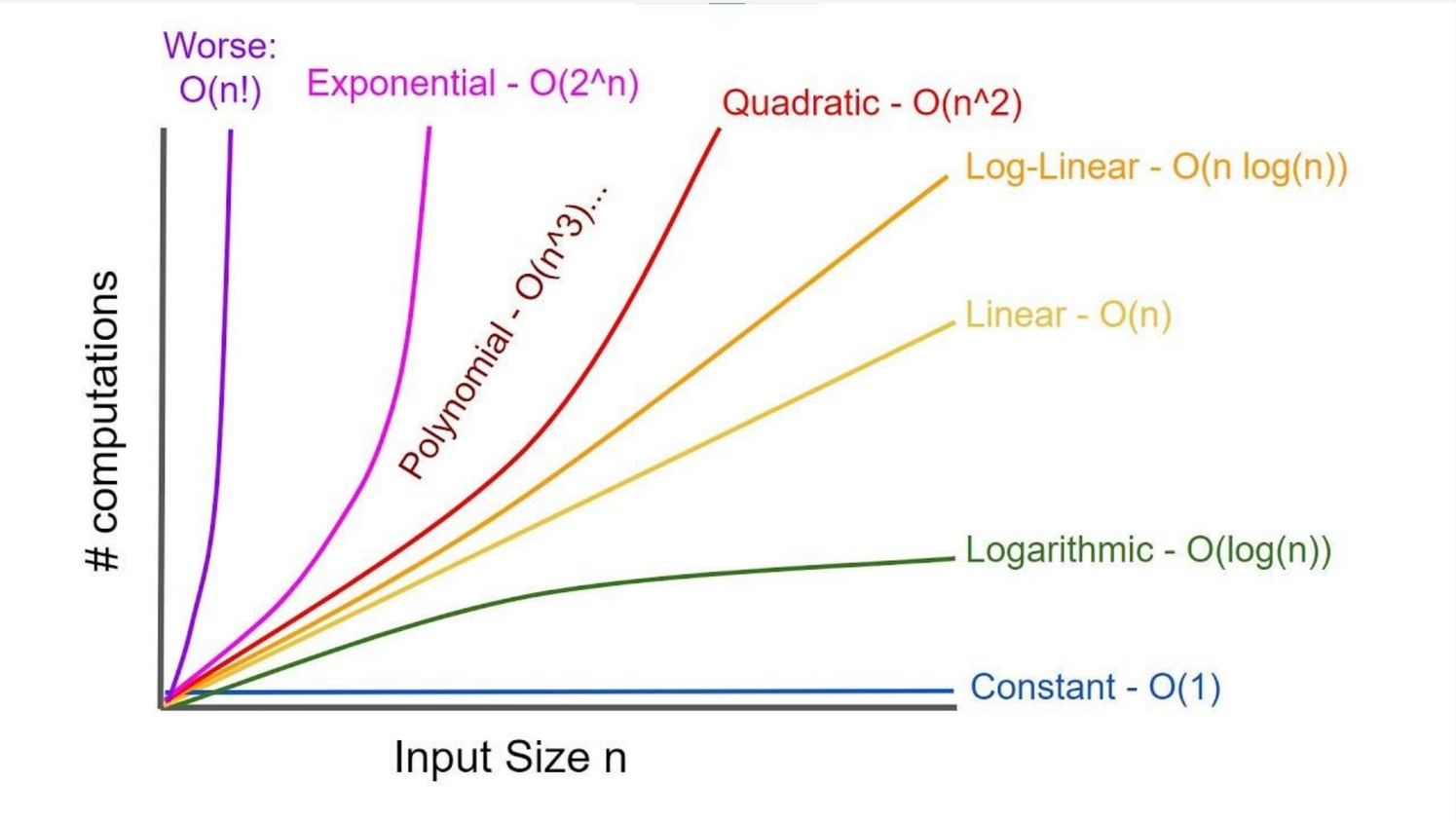
For example:

$$T_1(n) = An + B$$

Linear Time - O(n)

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:

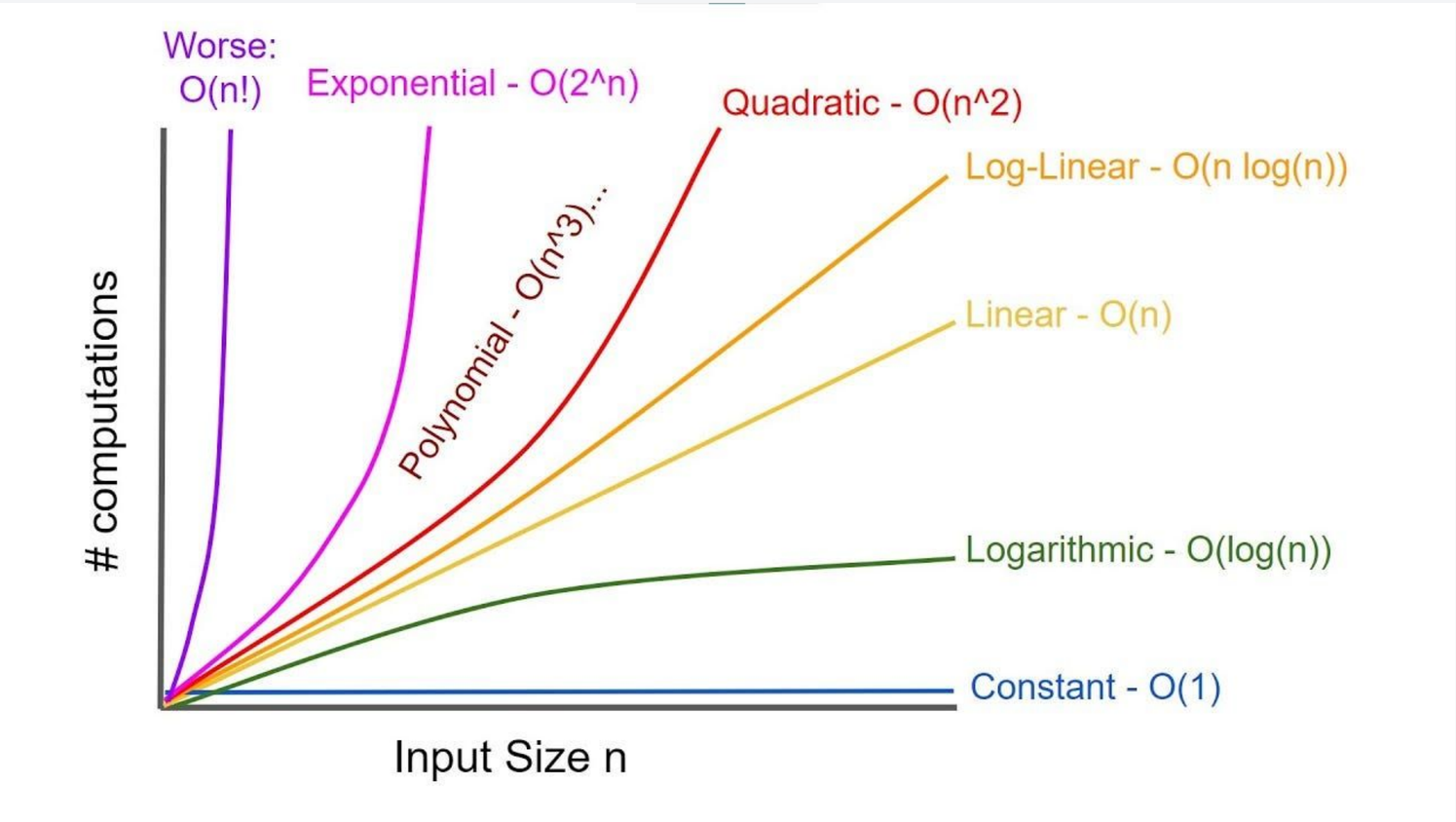


For example:

$$T_2(n) = Cn^2 + Dn + E$$

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:



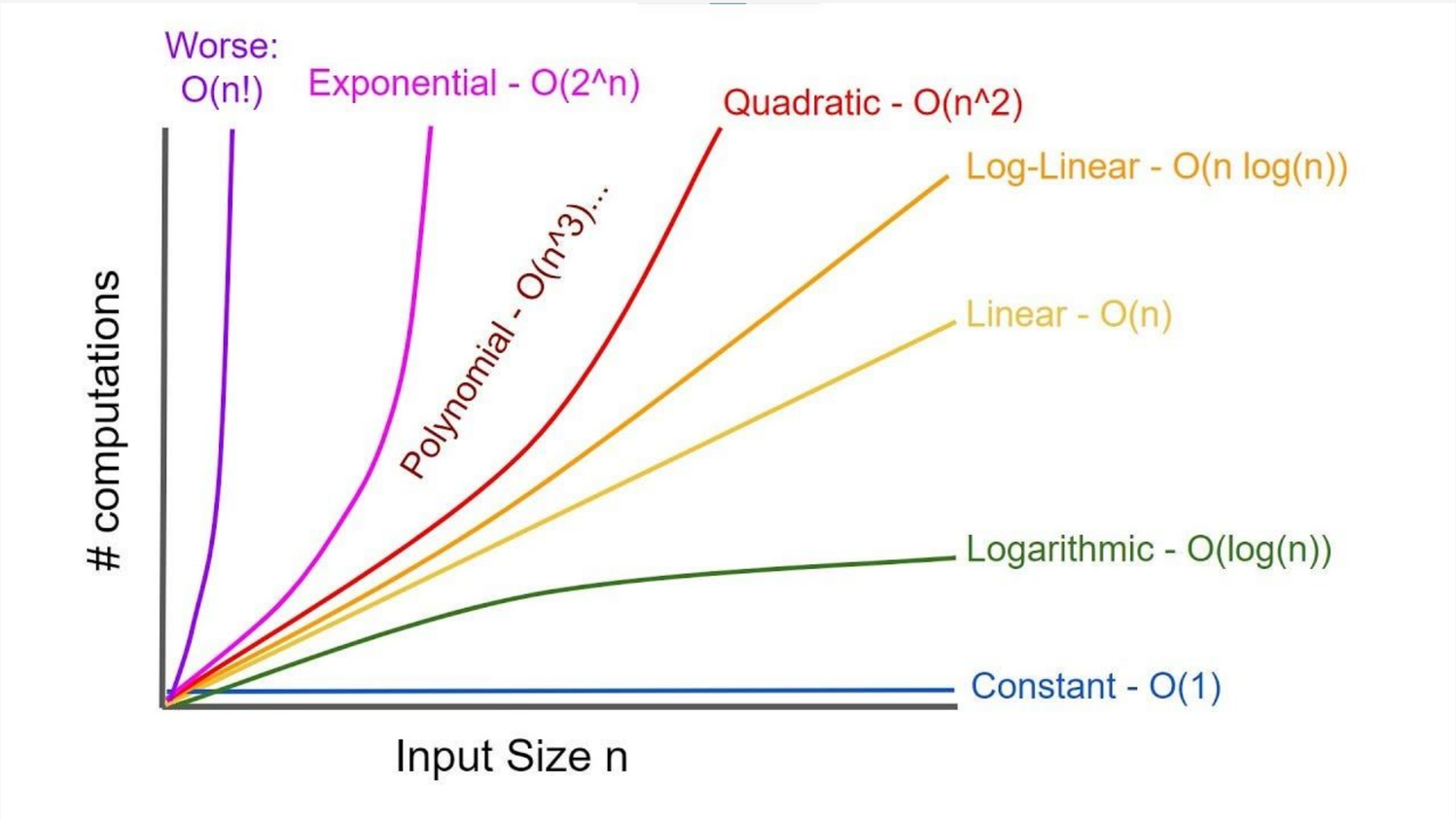
For example:

$$T_2(n) = Cn^2 + Dn + E$$

Quadratic Time - $O(n^2)$

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:

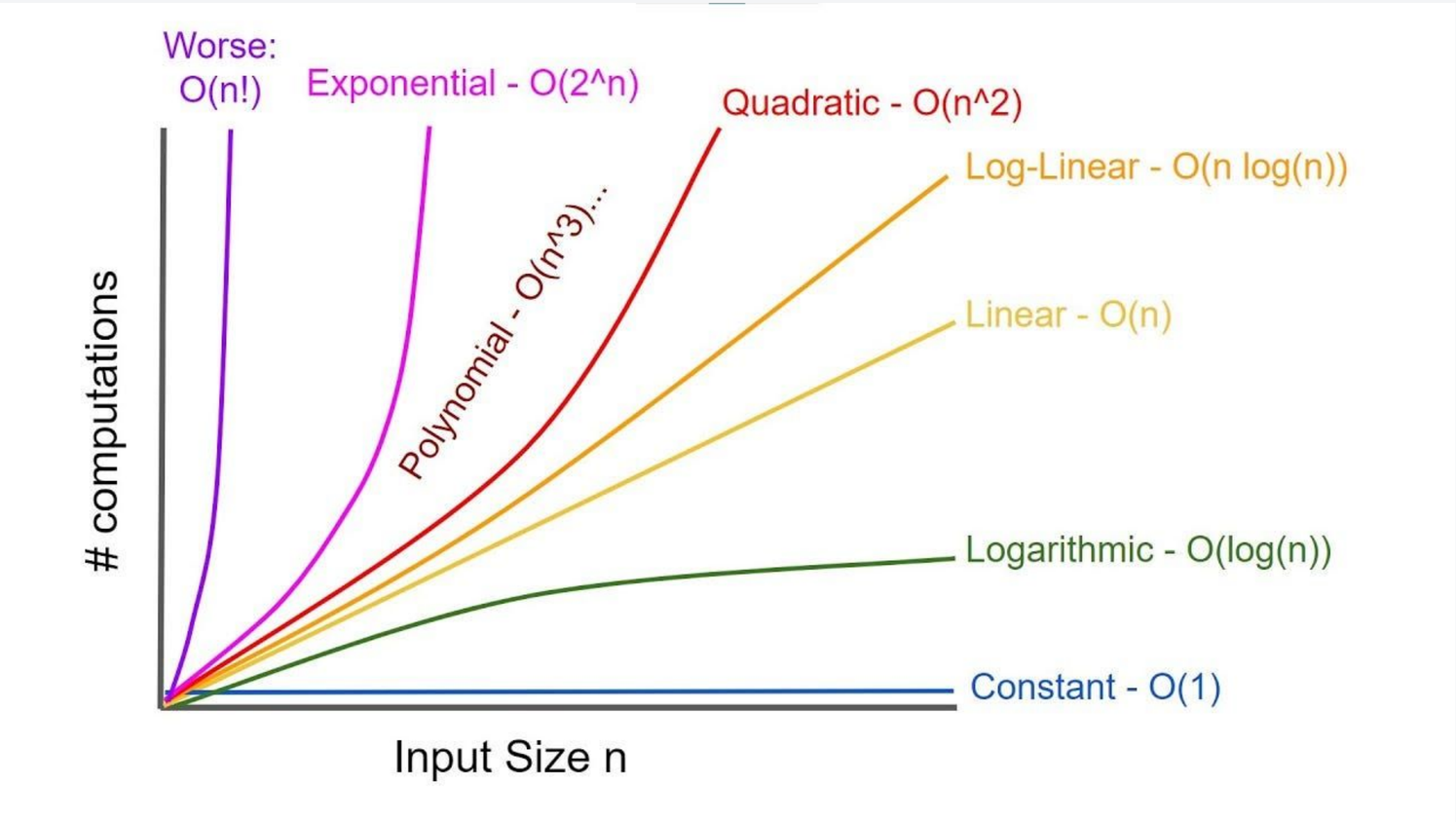


For example:

$$T_3(n) = F \cdot nlog_2 n + Gn + H$$

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:



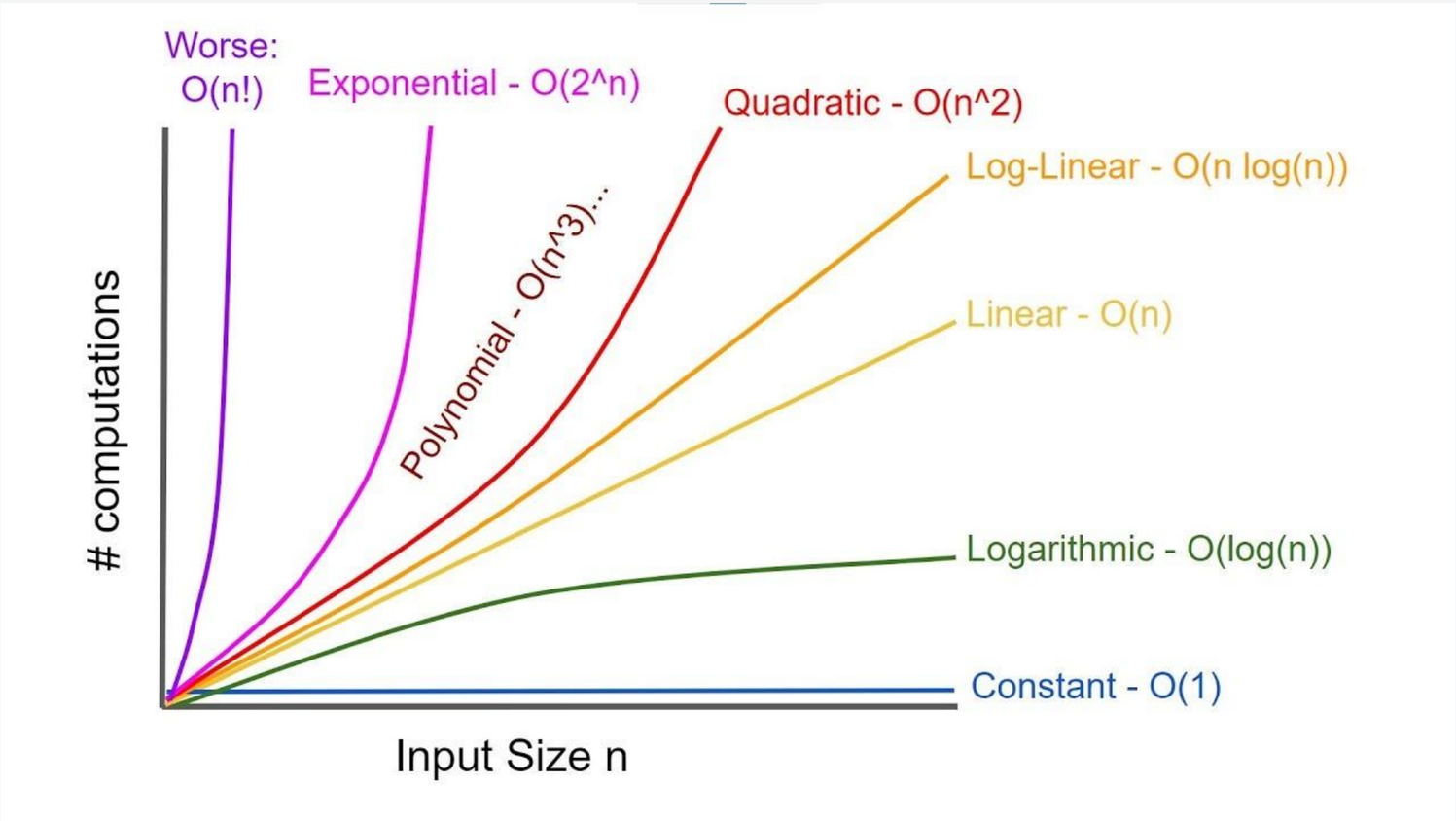
For example:

$$T_3(n) = F \cdot nlog_2n + Gn + H$$

Log-linear Time - O(nlog(n))

Asymptotic Behavior – Runtime Categories

Since only the fastest growing term is considered when measuring the runtime for functions, we usually categorize them into the following:



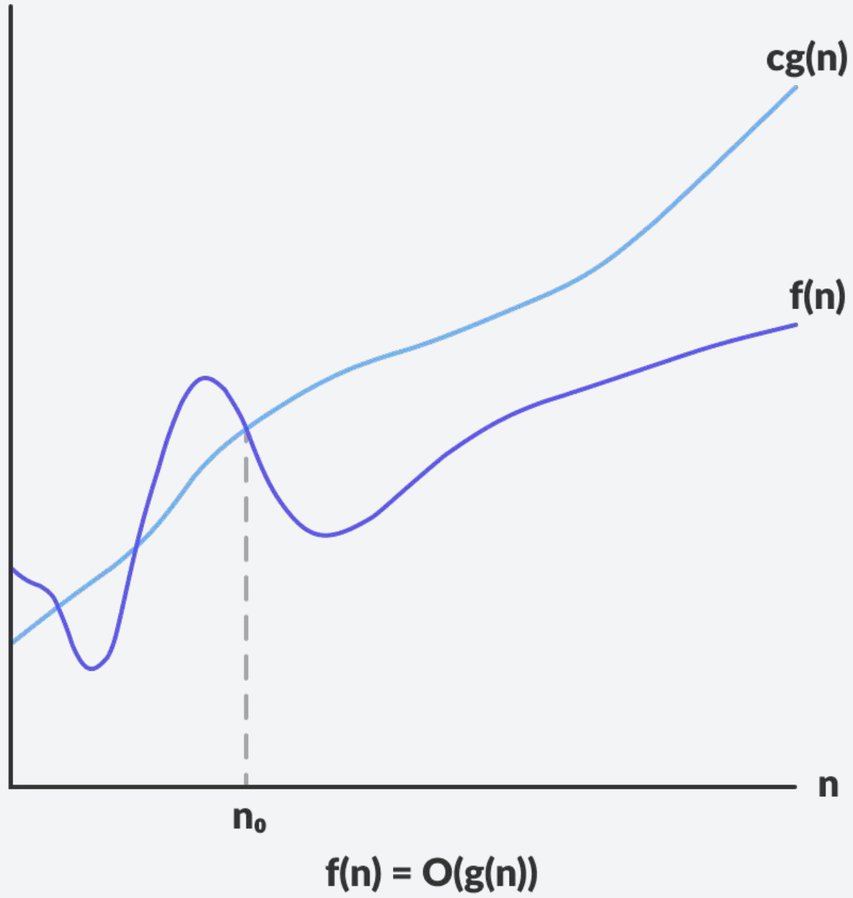
Only the fastest growing term matters!

- All about algorithms
- Runtime Analysis
- Asymptotic Notations
- **Big-O**

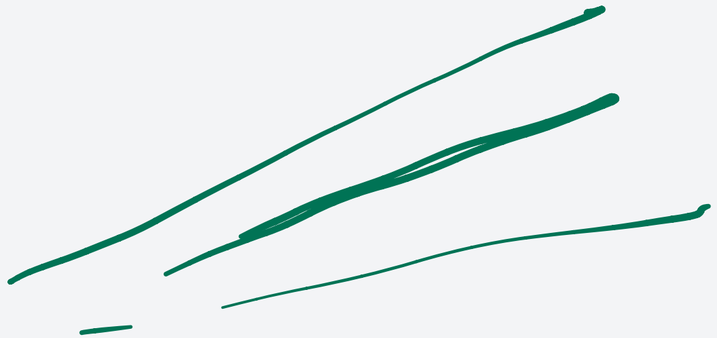
Big-O Notation

O - worst (upper bound)

Ω - lower bound (best)

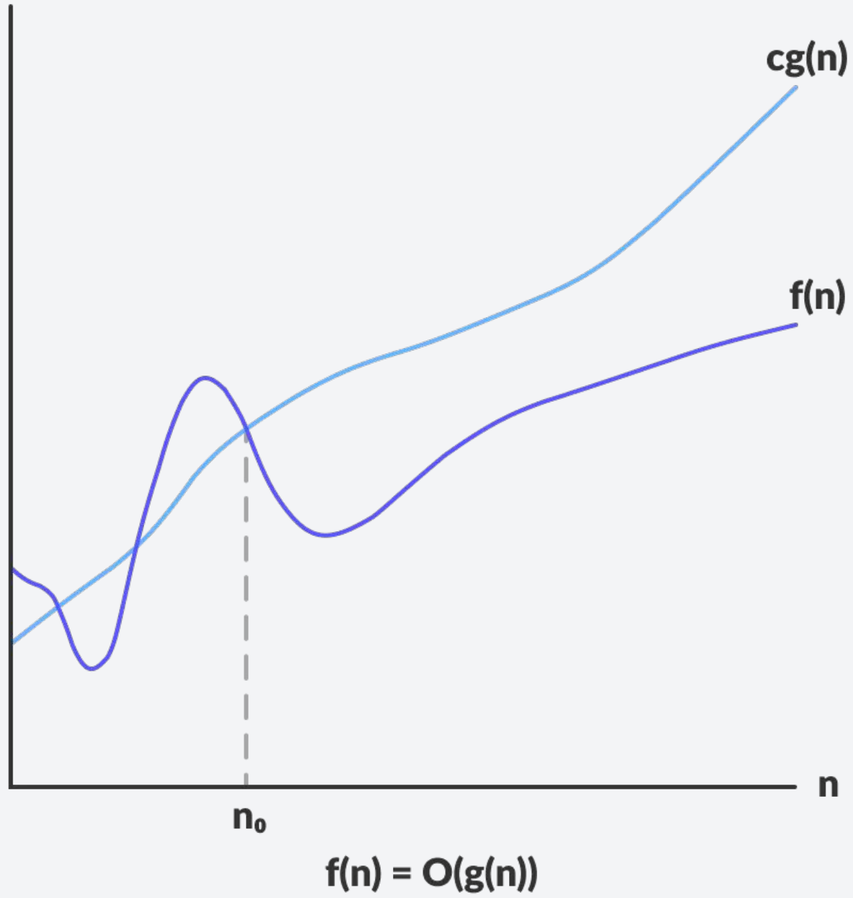


- The Big-O notation is defined as the **upper bound** when analyzing an algorithm.
  - Worst case complexity



Big-O Notation

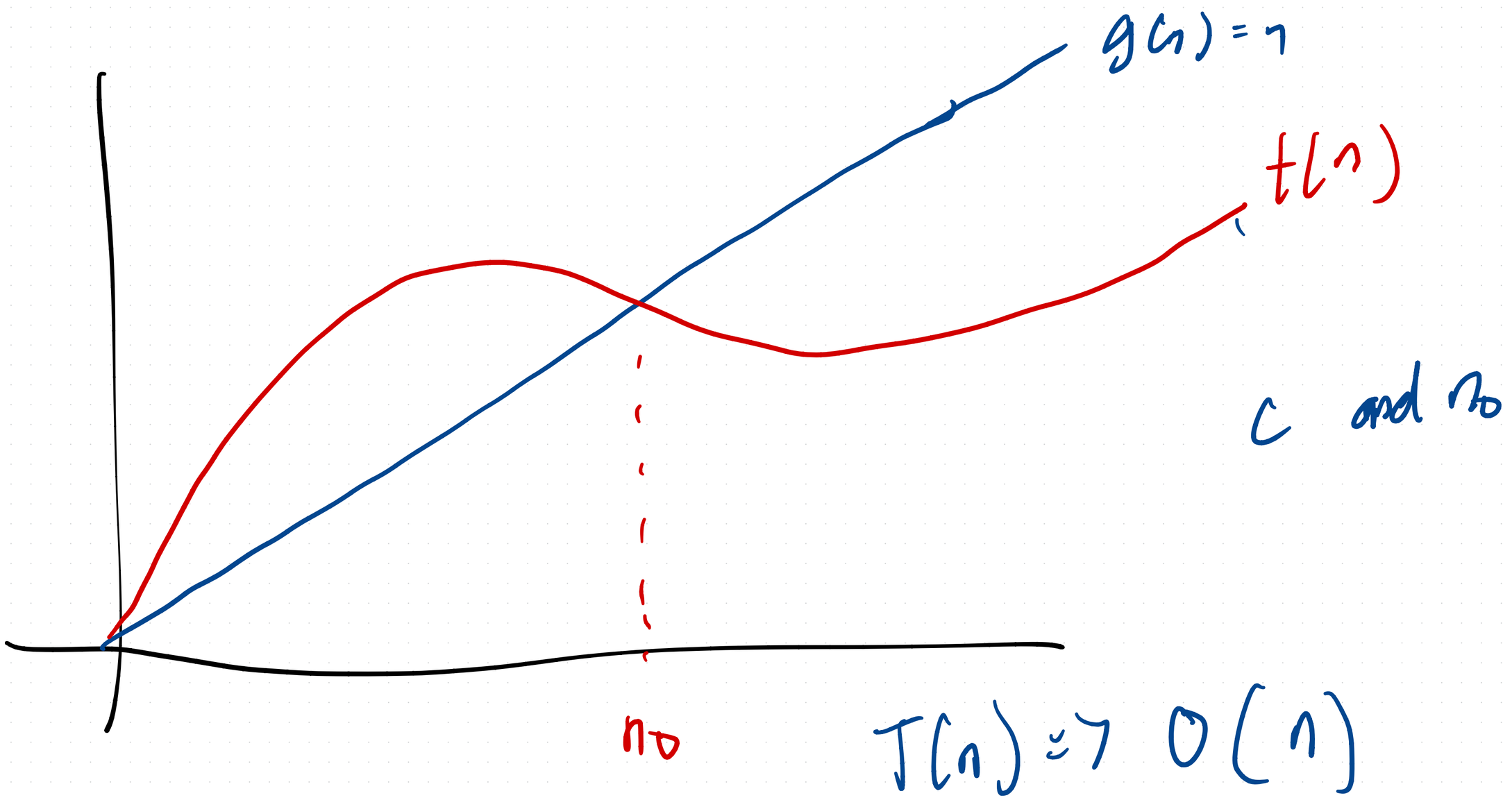
g(n) can be $1, \log n, n \log n, n, n^2, \ldots$



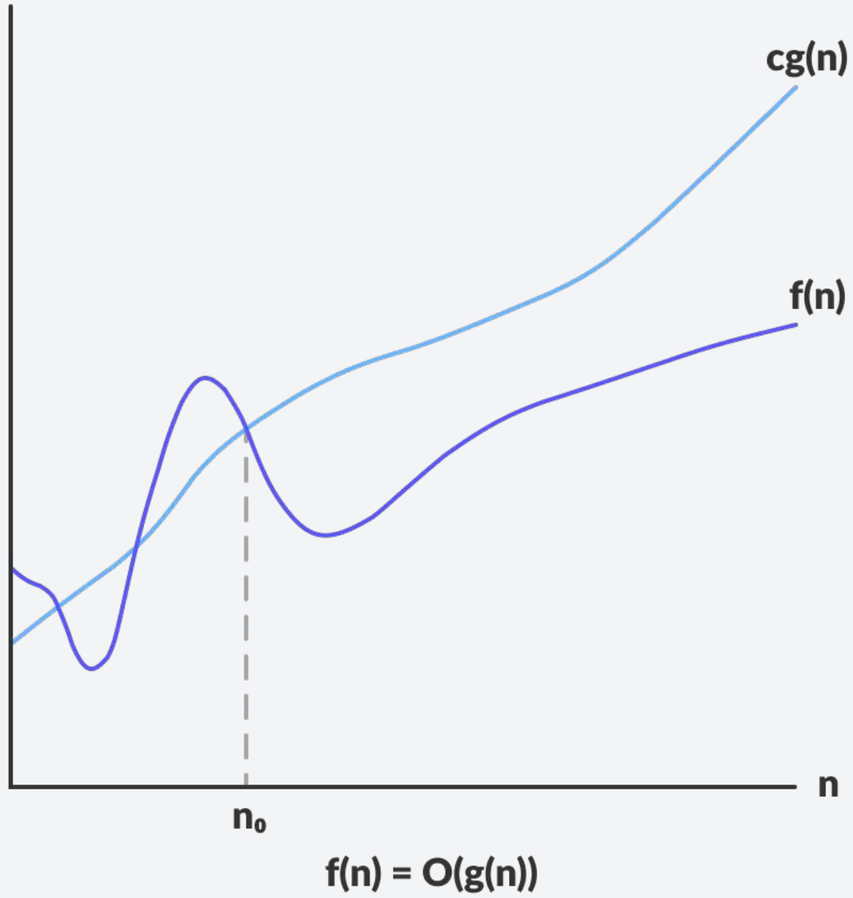
- The Big-O notation is defined as the **upper bound** when analyzing an algorithm.
  - Worst case complexity
- Mathematically speaking, a function $t(n)$ is said to be $O(g(n))$ if there exists a constant C such that:

$$0 \leq t(n) \leq cg(n) \quad \forall \; n \geq n_0$$

↳ runtime of your code


g(n) = n
t(n)
c and n0
n0
T(n) ∈ O(n)

Big-O Notation



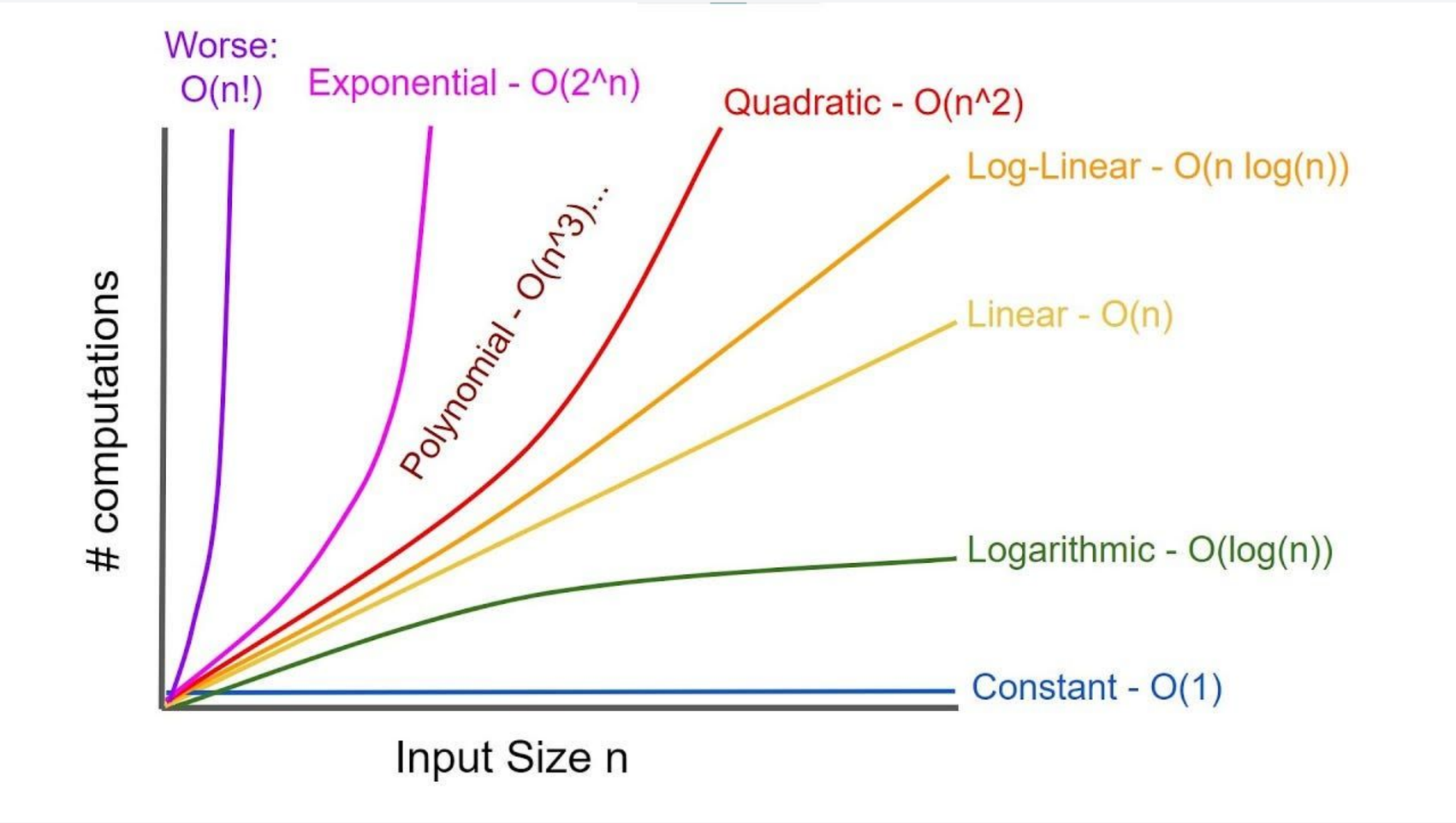
- The Big-O notation is defined as the **upper bound** when analyzing an algorithm.
  - Worst case complexity
- Mathematically speaking, a function $t(n)$ is said to be $O(g(n))$ if there exists a constant C such that:

$$0 \leq t(n) \leq cg(n) \quad \forall \; n \geq n_0$$

- This means that $t(n)$ does not grow faster than any multiples of $g(n)$ for large values of n.

Some known Big-O Complexity

Notice now why only the fastest growing term matters?



How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

1. The runtime of any operator is O(1). This includes:
    - Retrieval, Storage, Assignment of Variables
    - Arithmetic, Logical, Relational, and Bitwise Operators
    - Memory allocation and deallocation

=

+, -, *, /, ||, &&, &, |

delete, new[]

For a code that runs only a fixed number of operations (not dependent on input size), they run at constant time, O(n).

```
// Swapping values of a and b
int tmp = a;    // O(1)
a = b;          // O(1)
b = tmp;        // O(1)
```

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

2. Analyze each operation by blocks.

- Recall, $T(n) + S(n) = O(max(f(n), g(n))$.

```
int get_sum(const int* arr,int n) {
   int total = 0;
   for(int i = 0; i < n; i++) {
      total += arr[i];
   }
   return total;
}
```

O(1)

O(n)

O(1)

$O(1+n+1) = O(n+2) = O(n)$ //

What's the worst case complexity?

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

2. Analyze each operation by blocks.

- Recall, $T(n) + S(n) = O(max(f(n), g(n))$.

```
int get_sum(const int* arr,int n) {
  int total = 0;
  for(int i = 0; i < n; i++) {
    total += arr[i];
  }
  return total;
}
```

The for loop slows the program!

O(1)

O(n)

O(1)

$$O(1) + O(n) + O(1) = O(max(1,n,1)) = O(n)$$

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

3. Identify the algorithm's most basic operation first then work your way up.
   - Start at the inner loops first.

For If-else statements:

```
if ( condition ) {
    // true body
} else {
    // false body
}
```

O(1)

O(n)

This is usually O(1)

$$\textit{Total Runtime} = \textit{Runtime for checking condition} + \textit{Runtime for executing the body}$$

Depends on the function body

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

3. Identify the algorithm's most basic operation first then work your way up.
   - Start at the inner loops first.

Loop Statements:

```
int sum = 0;
for( int i = 0; i < n; ++i ) {
    for( int j = 0; j < m; ++j ) {
        sum += 1; //O(1)
    }
}
```

- Look at the inner loop first.
- It has a for loop that runs for m times.
- Thus, the inner loop has O(m).

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

3. Identify the algorithm's most basic operation first then work your way up.
   - Start at the inner loops first.

Loop Statements:

```
int sum = 0;
for( int i = 0; i < n; ++i ) {
    for( int j = 0; j < m; ++j ) {
        sum += 1; //O(1)
    }
}
```

O(m)

- The outer loop repeats for n times.
- Additionally, the inner loop runs for m times.
- Thus, O(m × n).

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

3. Identify the algorithm's most basic operation first then work your way up.
   - Start at the inner loops first.

Loop Statements:

```
int sum = 0;
for( int i = 0; i < n; ++i ) {
    for( int j = 0; j < m; ++j ) {
        sum += 1; //O(1)
    }
}
```

O(m)

O(m×n) = O(nm)

How to analyze using Big-O?

When determining the worst case complexity of the runtime, we follow some basic rules:

3. Identify the algorithm’s most basic operation first then work your way up.
    - Start at the inner loops first.

Recursive Functions??

Properties of the Big-O Notation

1. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n) + S(n) = O(f(n)+g(n)) = O(max(f(n), g(n))$$

$T(n) = n$

$S(n) = n^2$

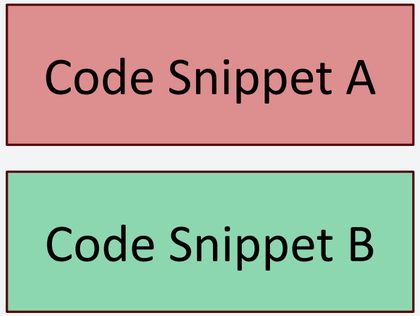
$\Rightarrow O(n + n^2) = O(n^2)$

Properties of the Big-O Notation

1. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n) + S(n) = O(f(n)+g(n)) = O(max(f(n), g(n))$$

Example: We have a code snippet A that has a runtime of $T(n)$ and code snippet B has a runtime of $S(n)$. Then, the runtime of executing the two code snippets sequentially is the maximum of their runtime.



Properties of the Big-O Notation

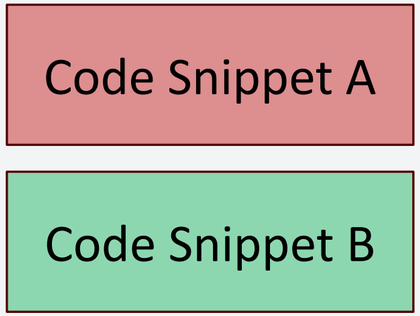
1. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n) + S(n) = O(f(n)+g(n)) = O(max(f(n), g(n))$$

Example: We have a code snippet A that has a runtime of $T(n)$ and code snippet B has a runtime of $S(n)$. Then, the runtime of executing the two code snippets sequentially is the maximum of their runtime.

$$T(n) + S(n) = O(max(f(n), g(n))$$

Complexity is dominated by the slower code snippet



Properties of the Big-O Notation

Code Snippet A

Code Snippet B

```
// Code Snippet A
printf(“Hello, world!);          // O(n) = 1

// Code Snippet B
int n;
for (int i = 0; i < n; i++);     // O(n) = n
```

Properties of the Big-O Notation

Code Snippet A

Code Snippet B

```
// Code Snippet A
printf(“Hello, world!);          // O(n) = 1

// Code Snippet B
int n;
for (int i = 0; i < n; i++);     // O(n) = n
```

$$T(n) + S(n) = O(n)+O(1) = O(n+1) = O(max(f(n), g(n)) \rightarrow O(n)$$

Do you see the similarity to the example we have when comparing runtimes?

Properties of the Big-O Notation

Code Snippet A

Code Snippet B

```
// Code Snippet A
printf(“Hello, world!);          // O(n) = 1

// Code Snippet B
int n;
for (int i = 0; i < n; i++);     // O(n) = n
```

$$T(n) + S(n) = O(n)+O(1) = O(n+1) = O(max(f(n), g(n)) \rightarrow O(n)$$

The time complexity of the code will still be dependent on Code Snippet B, the greater term.

Properties of the Big-O Notation

2. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n)S(n) = O(f(n)g(n))$$

Properties of the Big-O Notation

2. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n)S(n) = O(f(n)g(n))$$

Example: Similar example as the one we used previously, but this time, we run code snippet B inside code snippet A.

```
function A() {
    \\ do stuff
}
```

```
function B(n) {
   for(i = 0; i < g(n); i++)
  A();
}
```

Properties of the Big-O Notation

2. If we have $T(n) = O(f(n))$ and $S(n) = O(g(n))$. Then,

$$T(n)S(n) = O(f(n)g(n))$$

Example: Similar example as the one we used previously, but this time, we run code snippet B inside code snippet A.

$$T(n)S(n) = O(f(n)g(n))$$

This is particularly useful for nested loops and recursion

```
function A() {
    \\ do stuff
}
```

```
function B(n) {
   for(i = 0; i < g(n); i++)
  A();
}
```

Properties of the Big-O Notation

```
function A() {
    \\ do stuff
}
```

```
function B(n) {
   for(i = 0; i < g(n); i++)
  A();
}
```

```
// Code Snippet A
function A(){
   cout << “Hi” << endl;    // O(n) = 1
}
// Code Snippet B
int n;
for (int i = 0; i < n; i++){
   function A();  // This prints n times!
}
```

Properties of the Big-O Notation

```
function A() {
    \\ do stuff
}
```

```
function B(n) {
   for(i = 0; i < g(n); i++)
  A();
}
```

```
// Code Snippet A
function A(){
   cout << “Hi” << endl;    // O(n) = 1
}
// Code Snippet B
int n;
for (int i = 0; i < n; i++){
   function A();   // This prints n times!
}
```

$$T(n)S(n) = O(n)*O(1) = O(n*1) = O(n) \rightarrow O(f(n)g(n))$$

The time complexity gets multiplied.

This is how we get $O(n^2)$ 😉

Properties of the Big-O Notation

3. If the limit of f(n)/g(n) as n ➔ ∞ is **finite**, then $f(n) = O(g(n))$.

$\lim_{n \to \infty} \frac{T(n)}{g(n)} < \infty$, then $T(n) = O(g(n))$

ex. $T(n) = 2n$. Is $T(n) = O(n)$

$g(n) = n$

$\lim_{n \to \infty} \frac{2n}{n} = 2 \Rightarrow T(n) = O(n)$

Properties of the Big-O Notation

3. If the limit of f(n)/g(n) as n ➔ ∞ is **finite**, then $f(n) = O(g(n))$.
   - Recall, our mathematical definition of big-O.

$$T(n) = O(f(n)) \iff \exists\, c \in \mathbb{R}\ and\ \exists n_0 \in \mathbb{N}\ such\ that\ T(n) \leq c \cdot f(n)$$
$$\forall n \geq n_0$$

$$\exists\, c_1, c_2 \in \mathbb{R}\ and\ \exists n_1, n_2 \in \mathbb{N}\ such\ that\ \underbrace{T(n) \leq c_1 \cdot f(n) \leq c_1 c_2 g(n)}_{T(n) = O(g(n))}$$
$$\forall n \geq \max(n_1, n_2)$$

Properties of the Big-O Notation

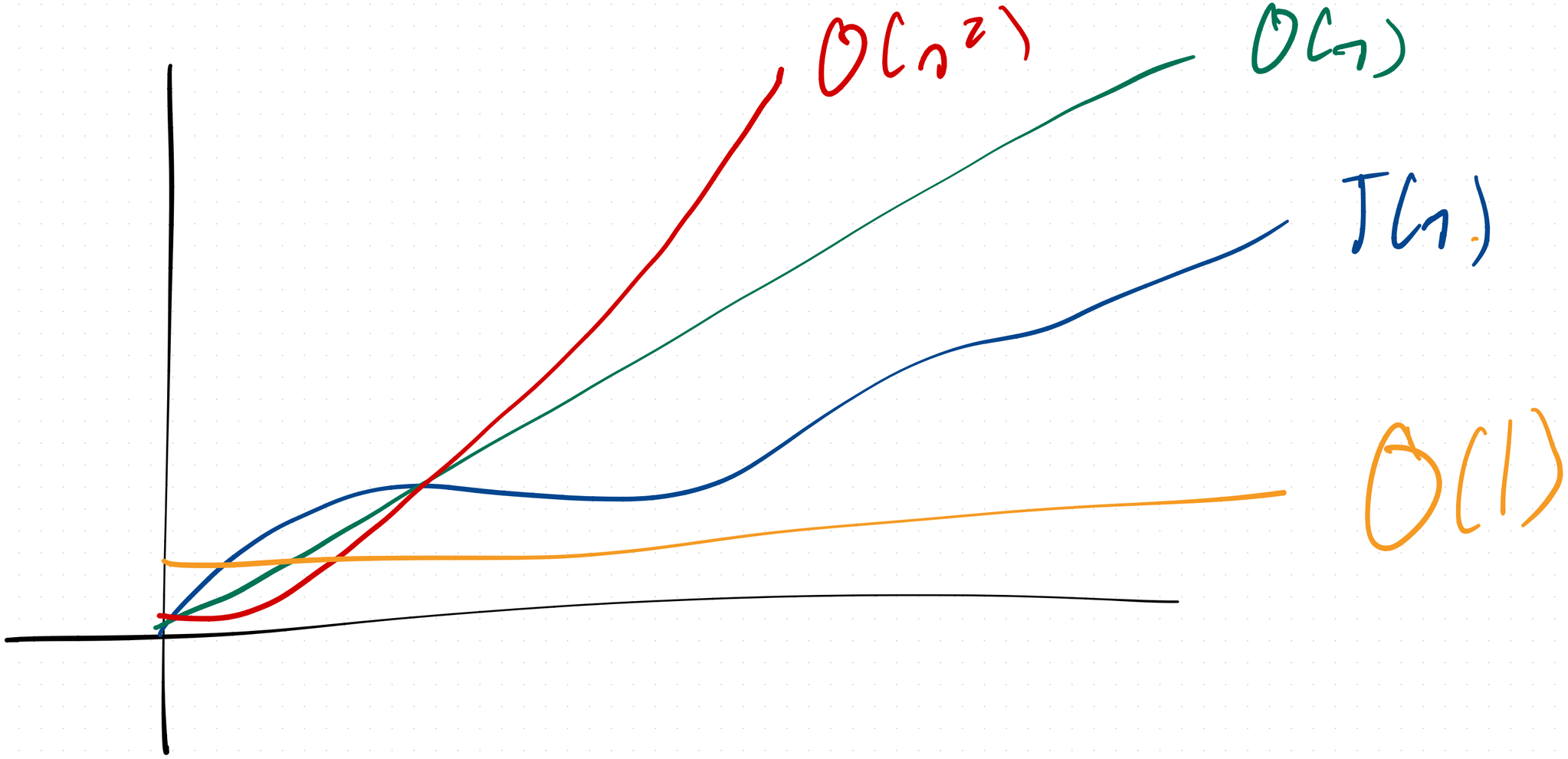
3. If the limit of f(n)/g(n) as n ➔ ∞ is **finite**, then $f(n) = O(g(n))$.
   - Recall, our mathematical definition of big-O.

$$T(n) = O(f(n)) \iff \exists\, c \in \mathbb{R}\ and\ \exists n_0 \in \mathbb{N}\ such\ that\ T(n) \leq c \cdot f(n)$$
$$\forall n \geq n_0$$

$$\exists\, c_1, c_2 \in \mathbb{R}\ and\ \exists n_1, n_2 \in \mathbb{N}\ such\ that\ \underbrace{T(n) \leq c_1 \cdot f(n) \leq c_1 c_2 g(n)}_{T(n) = O(g(n))}$$
$$\forall n \geq \max(n_1, n_2)$$

This also means that:

- All O(n) algorithms are also $O(n^2)$
- All $O(n^2)$ algorithms are also $O(n^3)$
- So on so forth.


O(n²)
O(n)
T(n)
O(1)

Properties of the Big-O Notation

3. If the limit of f(n)/g(n) as n ➔ ∞ is **finite**, then $f(n) = O(g(n))$.
   - Recall, our mathematical definition of big-O.

$$T(n) = O(f(n)) \iff \exists\, c \in \mathbb{R} \text{ and } \exists n_0 \in \mathbb{N} \text{ such that } T(n) \leq c \cdot f(n) \quad \forall n \geq n_0$$

$$\exists\, c_1, c_2 \in \mathbb{R} \text{ and } \exists n_1, n_2 \in \mathbb{N} \text{ such that } \underbrace{T(n) \leq c_1 \cdot f(n) \leq c_1 c_2 g(n)}_{T(n) = O(g(n))} \quad \forall n \geq \max(n_1, n_2)$$

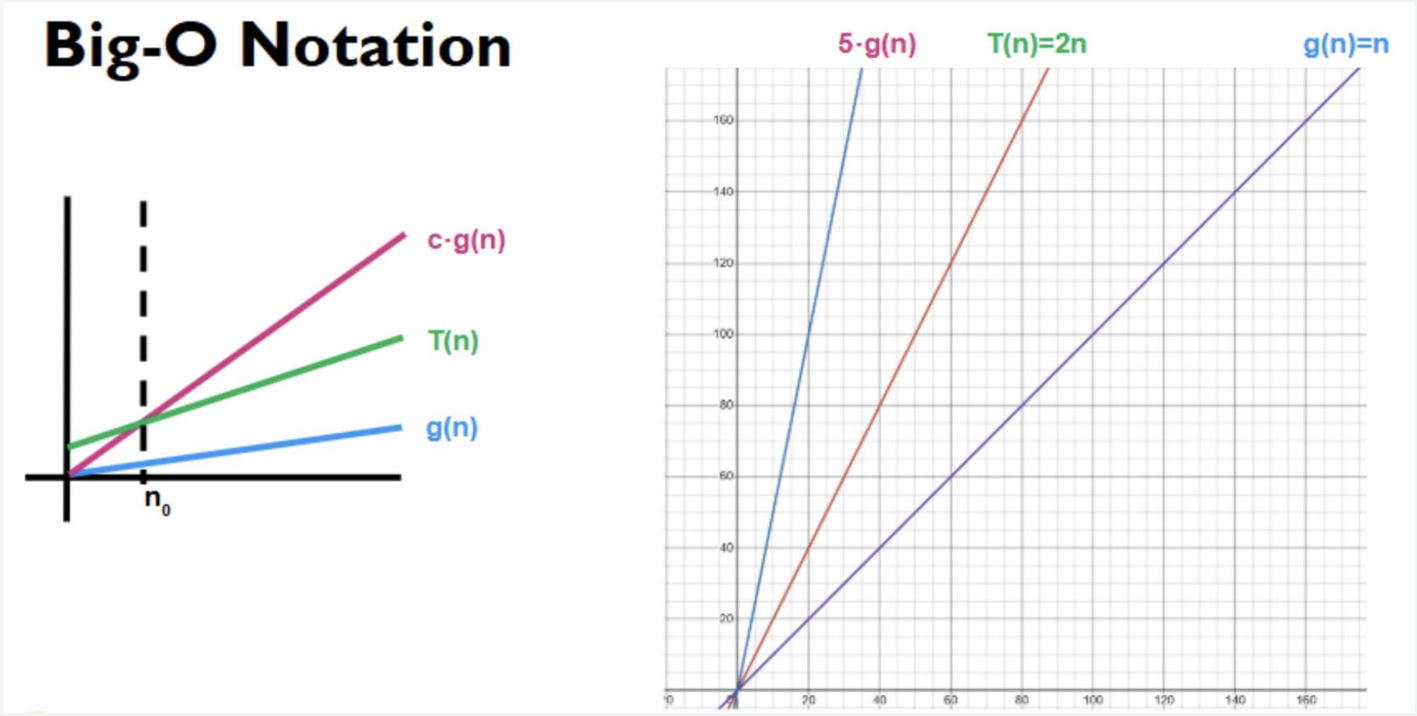
This also means that:

- All O(n) algorithms are also $O(n^2)$
- All $O(n^2)$ algorithms are also $O(n^3)$
- So on so forth.

But usually we use the tightest or most precise upper bound.

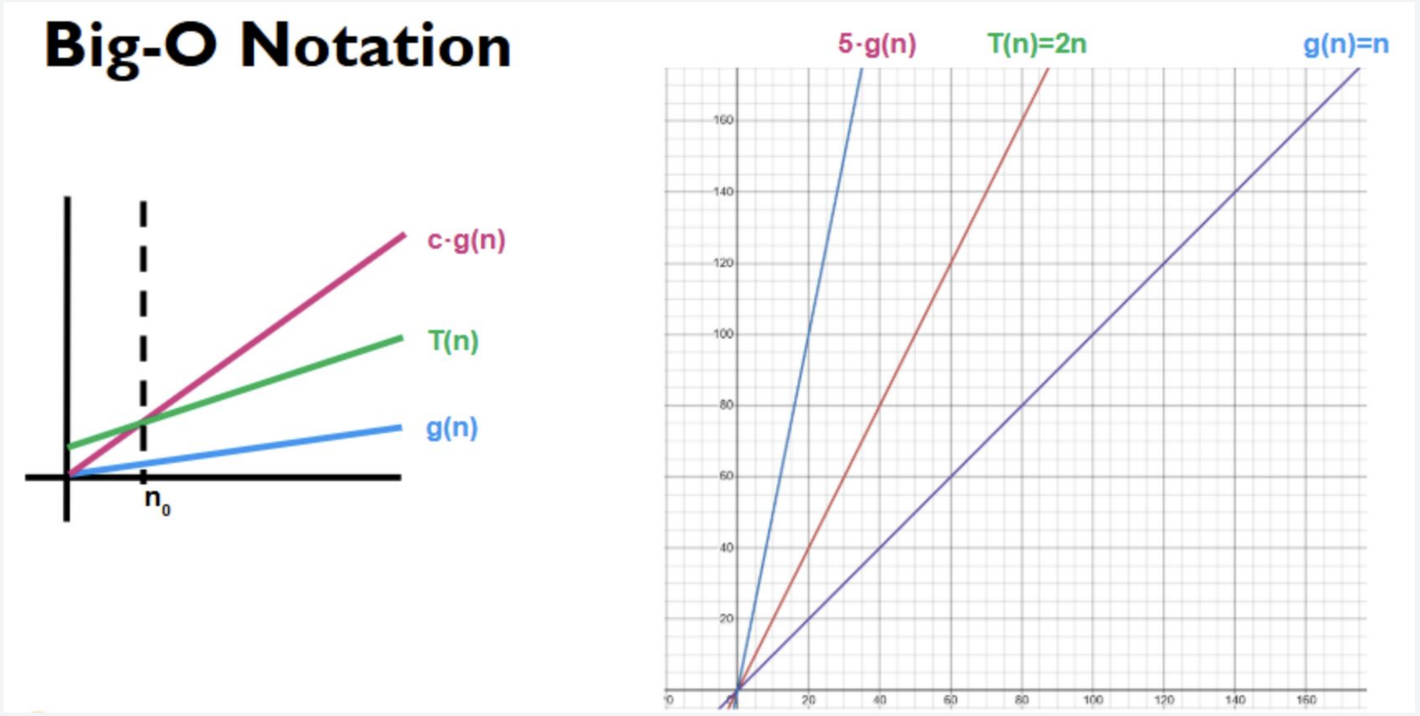
Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



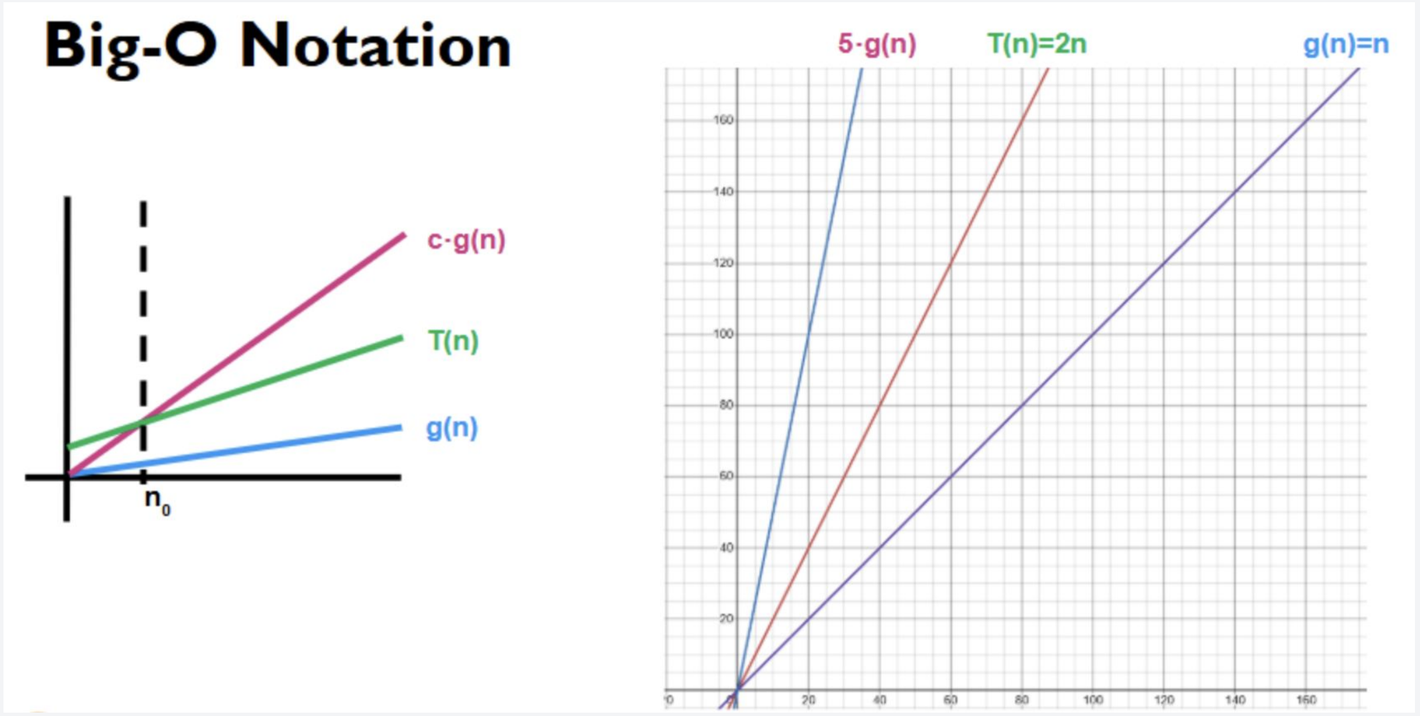
Let $g(n) = n$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n$,

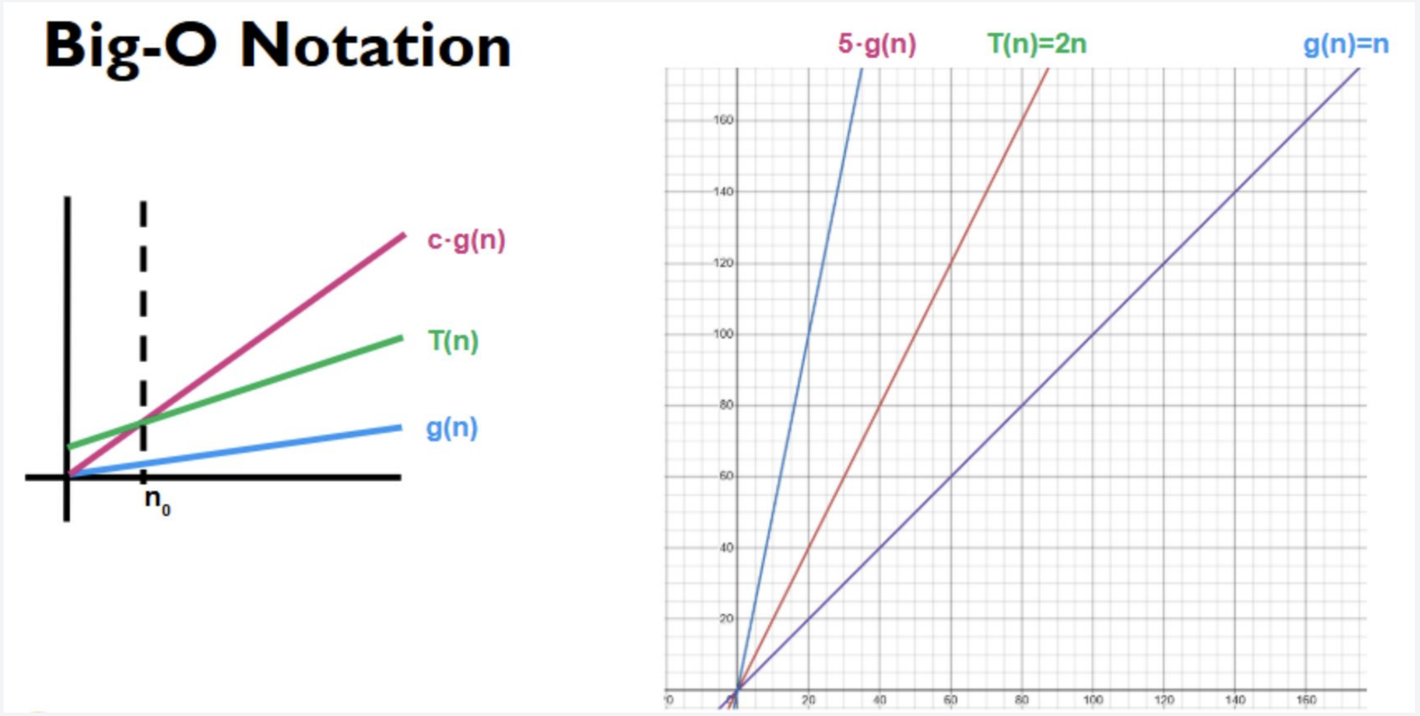
- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 5$

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 5$

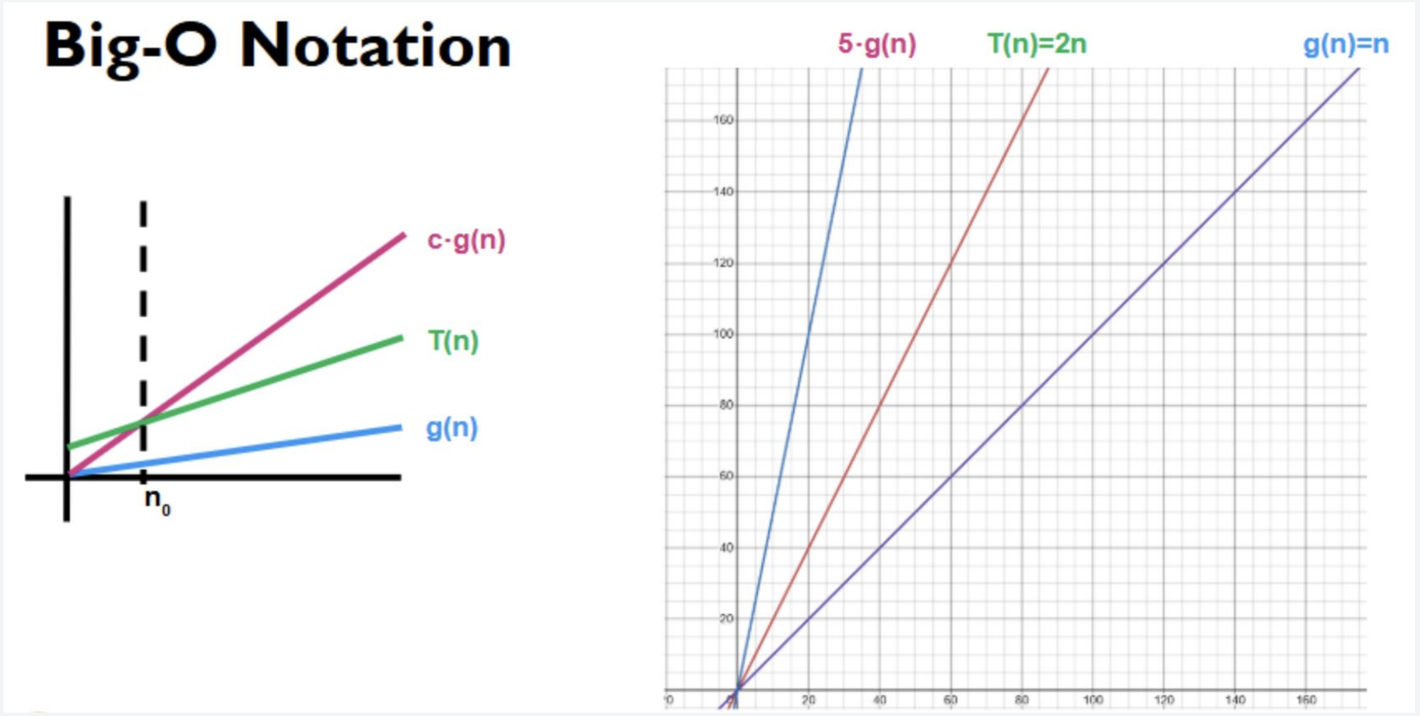
Alternatively:

$$\lim_{n\to\infty} \frac{T(n)}{g(n)} = \lim_{n\to\infty} \frac{2n}{n} = 2 < \infty$$

T(n) -> O(n)

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 5$

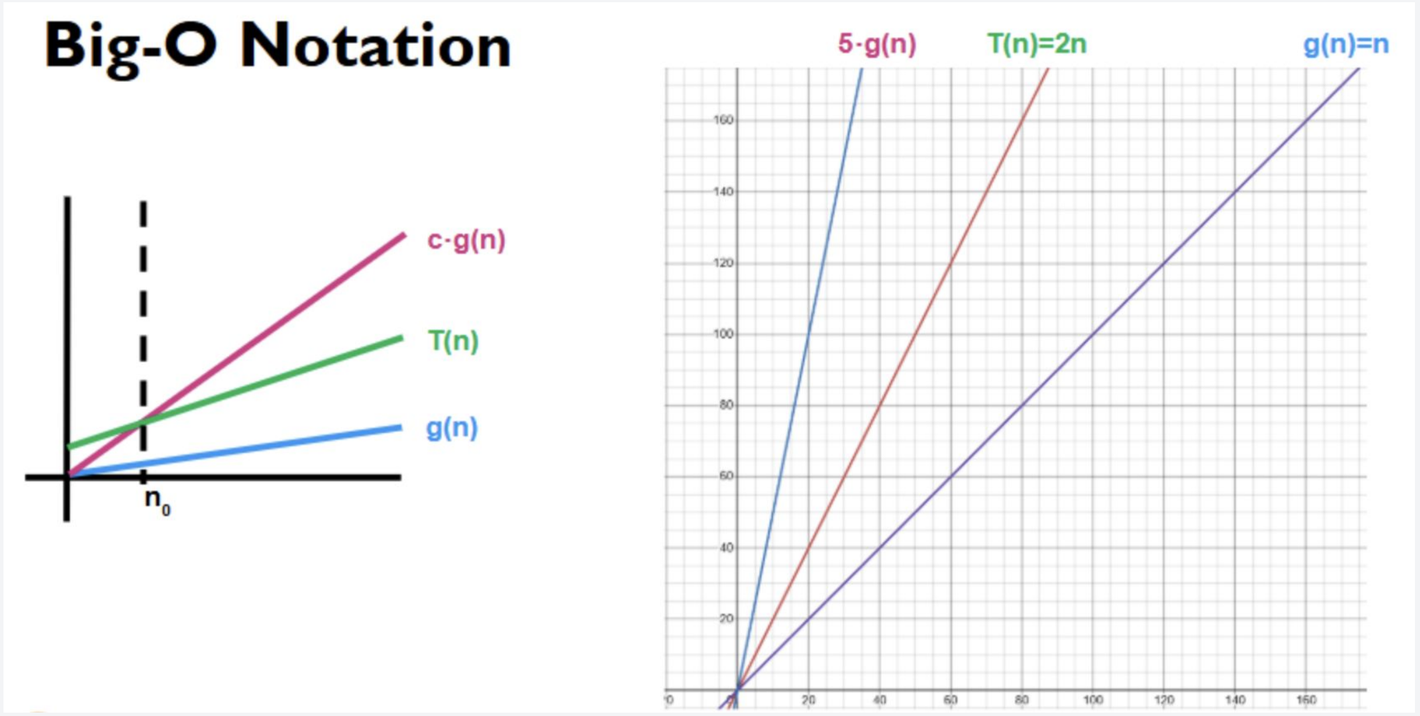
Alternatively:

$$\lim_{n\to\infty} \frac{T(n)}{g(n)} = \lim_{n\to\infty} \frac{2n}{n} = 2 \leq \infty$$

This means that T(n) = O(n)!

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 5$

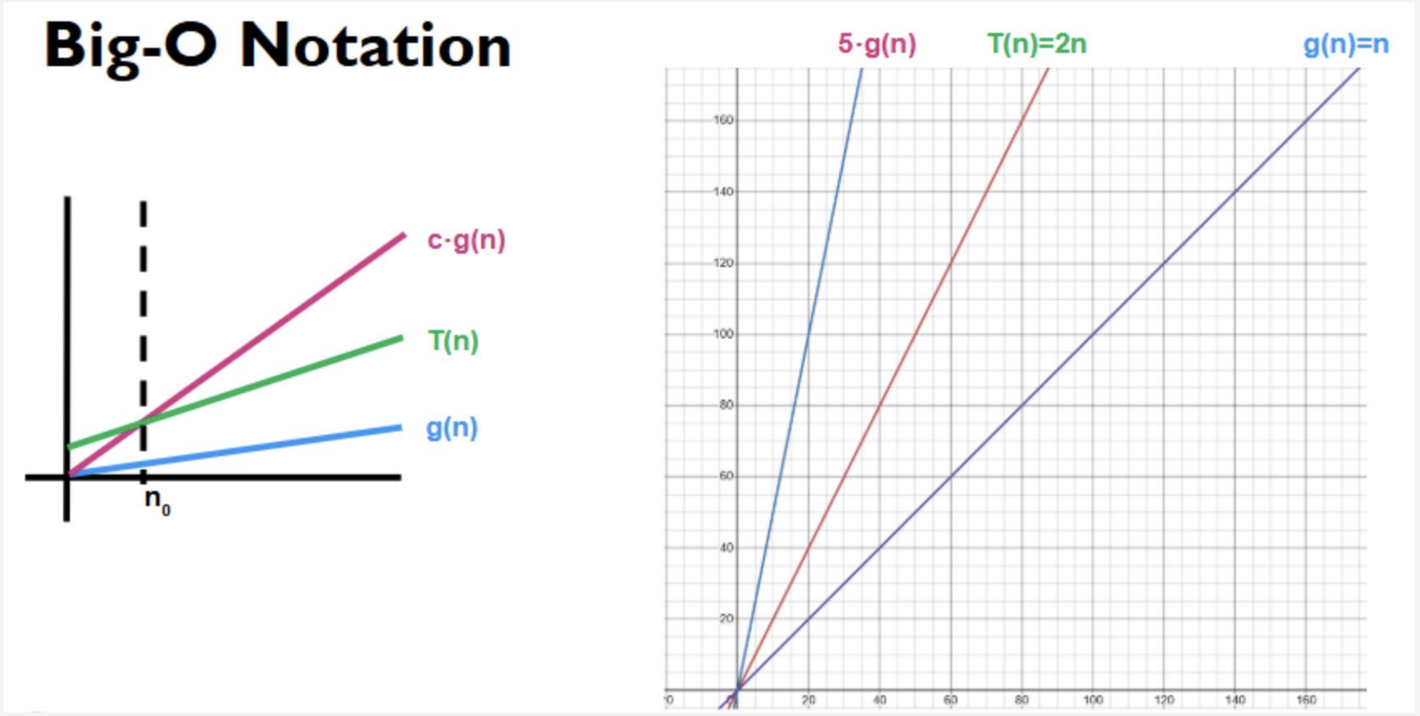
Alternatively:

$$\lim_{n\to\infty} \frac{T(n)}{g(n)} = \lim_{n\to\infty} \frac{2n}{n} = 2 \leq \infty$$

What if $g(n) = n^2$?

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n^2$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 1$

Alternatively:

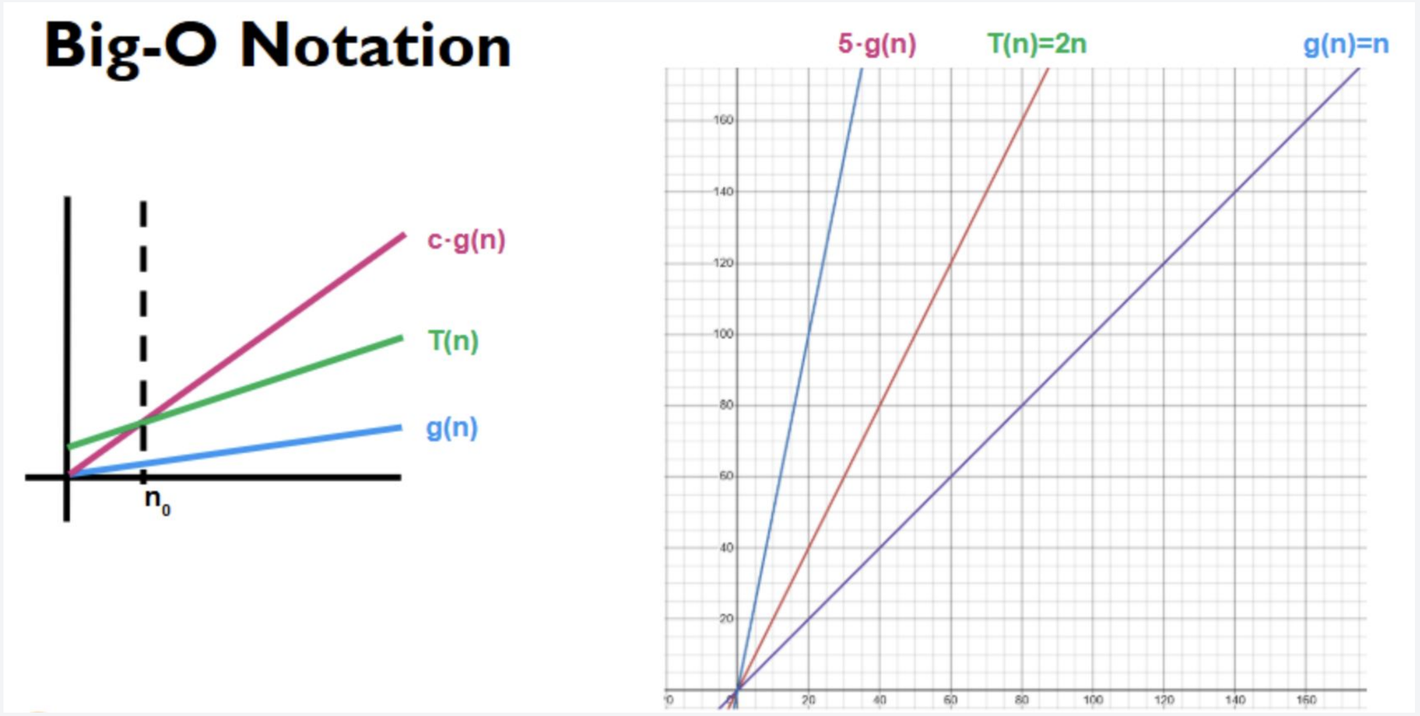
$$\lim_{n\to\infty} \frac{T(n)}{g(n)} = \lim_{n\to\infty} \frac{2n}{n^2} = 0 \leq \infty$$

$\lim_{n\to\infty} \frac{1}{n} = 0$

Same thing!

Properties of the Big-O Notation

For example, consider $T(n) = 2n$:



Let $g(n) = n^2$,

- There should exists a value n_0 and c such that:

$$T(n) \leq cg(n)$$

- $n_0 = 0$ and $c = 1$

Alternatively:

$$\lim_{n\to\infty} \frac{T(n)}{g(n)} = \lim_{n\to\infty} \frac{2n}{n^2} = 0 \leq \infty$$

However, as I said earlier, we will choose the tighter bound.

$f(n) = 2n \qquad g(n) = 1$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \lim_{n \to \infty} \frac{2n}{1} = \lim_{n \to \infty} 2n = \infty$$

$$f(n) \neq O(1)$$

Let’s do some examples!

Give me the time complexity of the given code

Item 1:

```
void example1(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            for (int k = j; k < n; k++) {
                cout << i << " " << j << " " << k << endl;
            }
        }
    }
}
```

Give me the time complexity of the given code

Item 1:

```
void example1(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            for (int k = j; k < n; k++) {
                cout << i << " " << j << " " << k << endl;
            }
        }
    }
}
```

Answer: O(n^3)

Give me the time complexity of the given code

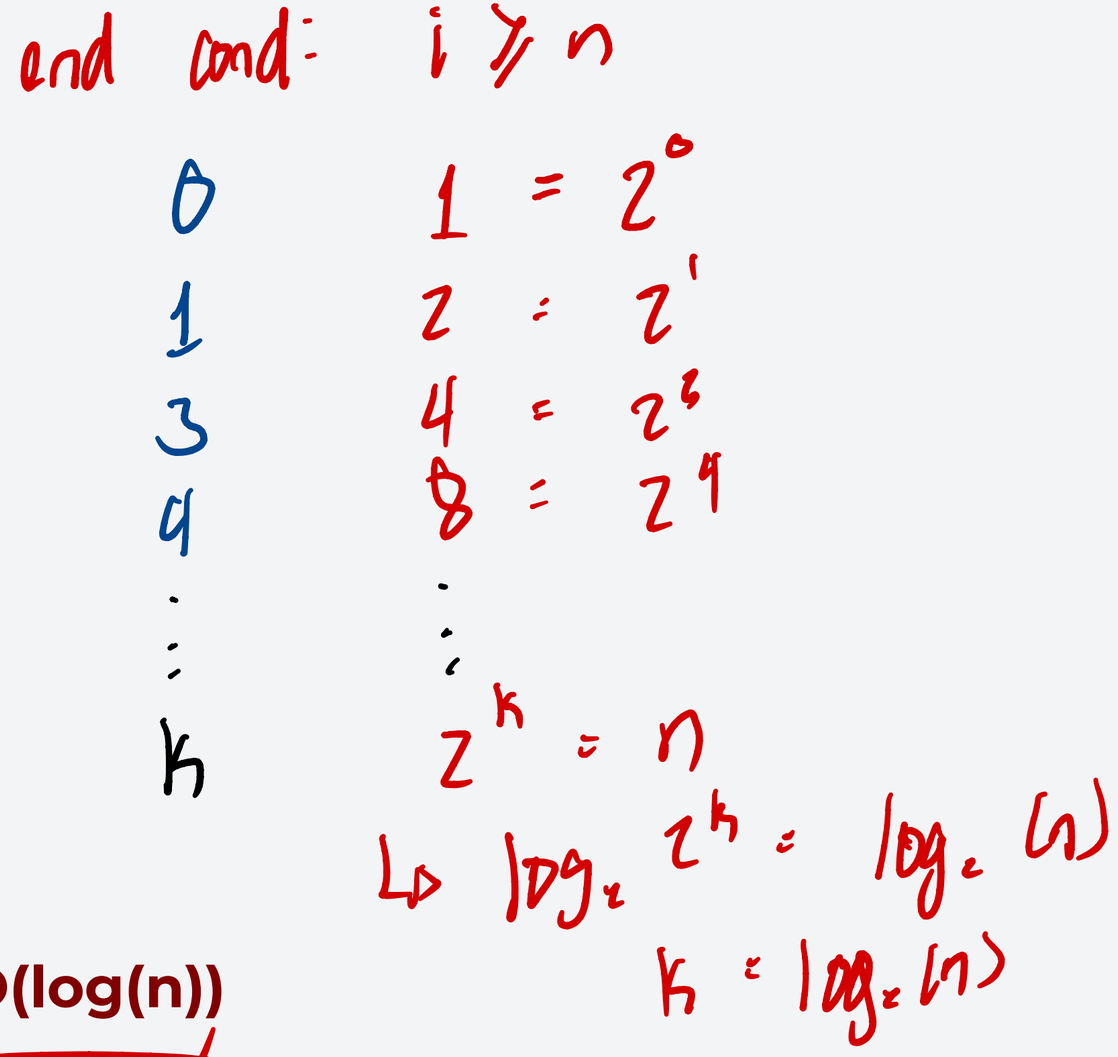
Item 2:

```
void example2(int n) {
    int i = 1;
    while (i < n) {
        cout << i << endl;
        i *= 2;
    }
}
```

Give me the time complexity of the given code

Item 2:

```
void example2(int n) {
    int i = 1;
    while (i < n) {
        cout << i << endl;
        i *= 2;
    }
}
```



Answer: O(log(n))

Give me the time complexity of the given code

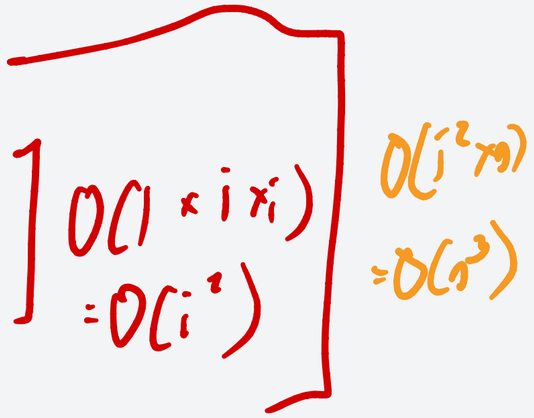
Item 3:

```
void example3(int n) {
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < i * i; j++) {
            cout << i << " " << j << endl;
        }
    }
}
```

Give me the time complexity of the given code

Item 3:

```
void example3(int n) {
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < i * i; j++) {
            cout << i << " " << j << endl;
        }
    }
}
```



Answer: $O(n^3)$

Give me the time complexity of the given code

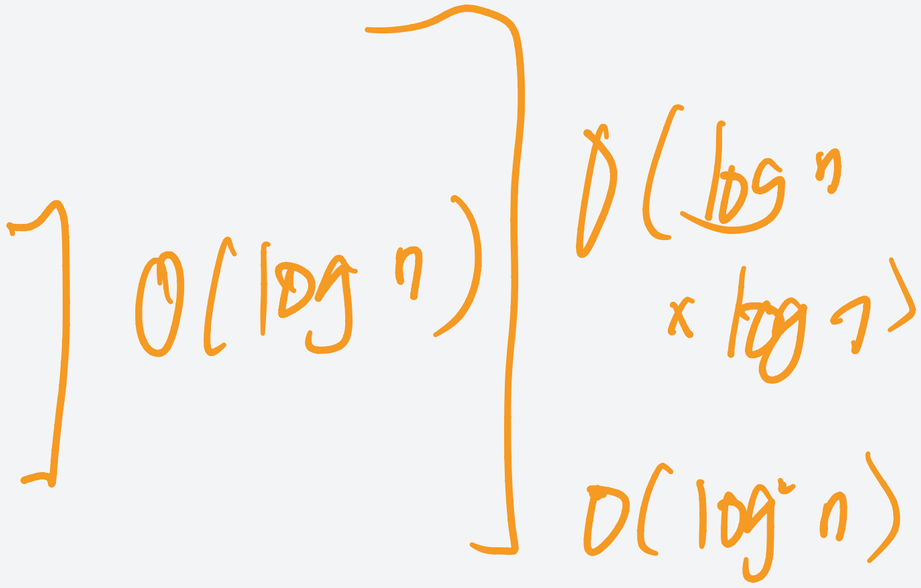
Item 4:

```
void example(int n) {
    int i = 1;
    while (i < n) {
        int j = 1;
        while (j < i) {
            cout << i << " " << j << endl;
            j *= 2;
        }
        i *= 3;
    }
}
```

Give me the time complexity of the given code

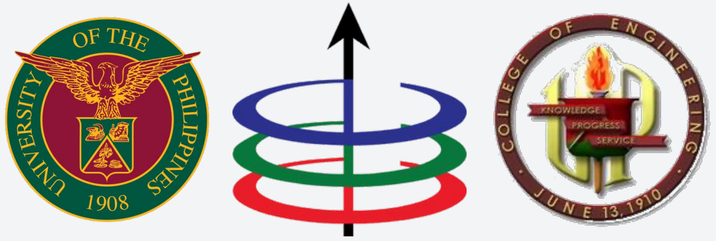
Item 4:

```
void example(int n) {
    int i = 1;
    while (i < n) {
        int j = 1;
        while (j < i) {
            cout << i << " " << j << endl;
            j *= 2;
        }
        i *= 3;
    }
}
```



Answer: O(log^2(n))

Any questions?



Lecture 2A
Algorithm Analysis - Big O

EEE 121 2s2526

Steven S. Sison